4. Vizualizacija podataka

Dve ključne biblioteke u Python programskom jeziku koje nam pomažu da kreiramo vizuelni prikaz za naše podatke u formi raznih vrsti grafikona su **Matplotlib** i **Seaborn**. Matplotlib je biblioteka za jednostavnije vizualizacije u podacima kreirana od strane Džona Hantera sa namerom da se kopiraju sposobnosti Matlab jezika za vizualizaciju podataka. Seaborn pruža naprednije koncepte u vizualizaciji. Počnimo od Matlab-a a kasnije iskoristimo Seaborn kako bismo zaista uspeli da uočimo značajne zakonitosti u podacima i koliko vizuelni prikaz podataka zaista može biti od značaja.

Sekcija 1 - Matplotlib

Počnimo od komande za import matplotlib biblioteke u kod:

In [1]:
```
import matplotlib.pyplot as plt
```

In [2]:
```
import warnings
warnings.filterwarnings('ignore')
```

Prođimo kroz jedan veoma jednostavan primer. Želimo da iscrtamo grafik za dva numpy niza. Jedan će predstavljati kordinate x dok će drugi predstavljati kordinate y ose u našim fiktivnim podacima.

In [3]:
```
import numpy as np
x = np.linspace(0,5,11)
y = x**2
```

Definišimo sada grafik za naše nizove:

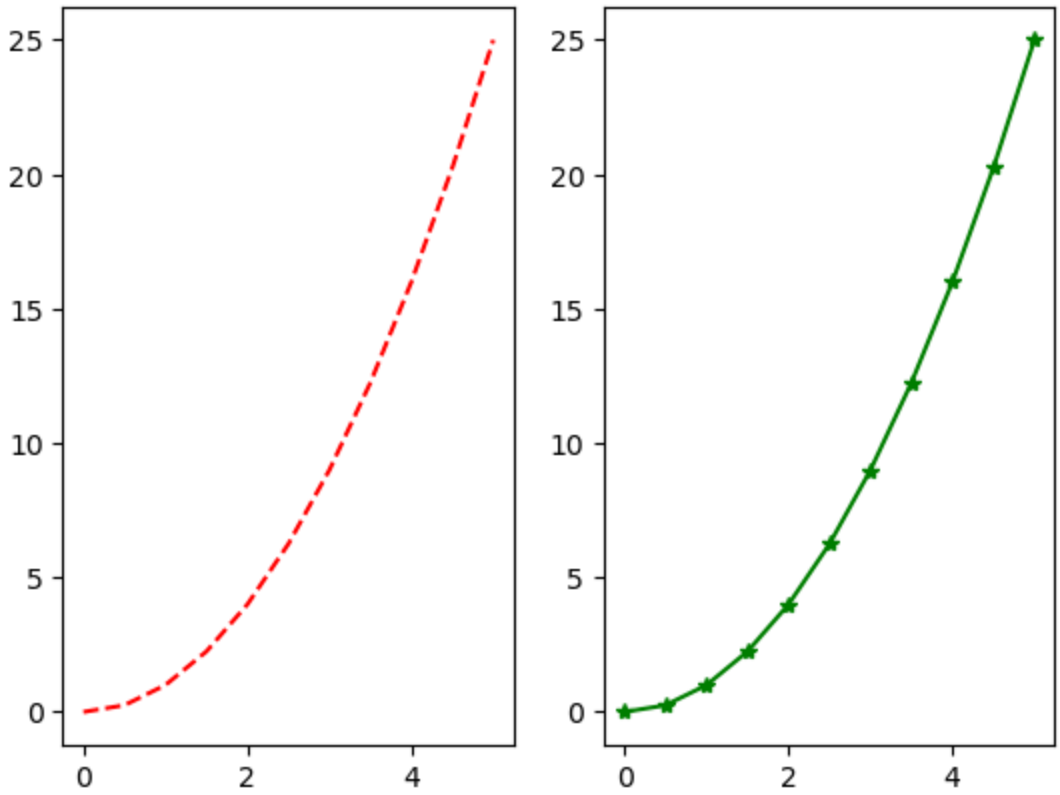
In [4]:
```
plt.plot(x,y,'r') # parametar r označava crvenu liniju (njenu boju)
plt.xlabel('Ovo je x osa')
plt.ylabel('Ovo je y osa')
plt.title('Ovo je naslov našeg grafika!')
plt.show()
```



Možemo kreirati i više grafika tako što ćemo koristiti funkciju `subplot`. Potpis funkcije `subplot` je:
`subplot(broj_redova_na_grafiku,broj_kolona_na_grafiku,broj_grafika)`

In [5]:
```
plt.subplot(1,2,1)
plt.plot(x,y,'r--')
plt.subplot(1,2,2)
plt.plot(x,y,'g*-')
```

Out[5]:
```
[<matplotlib.lines.Line2D at 0xa270b50>]
```



Matplotlib podržava metod rada koji podseća na objektno orijentisano programiranje. Moguće je da instanciramo objekat tipa `figure` i da nakon toga pozivamo metode ili atribute tog `figure` objekta.

In [6]:
```
fig = plt.figure()
axes = fig.add_axes([0.1,0.1,0.8,0.8])
axes.plot(x,y,'b')
axes.set_xlabel('Ovo je x osa')
axes.set_ylabel('Ovo je y osa')
axes.set_title('Ovo je naslov grafika')
```

Out[6]:
```
Text(0.5, 1.0, 'Ovo je naslov grafika')
```

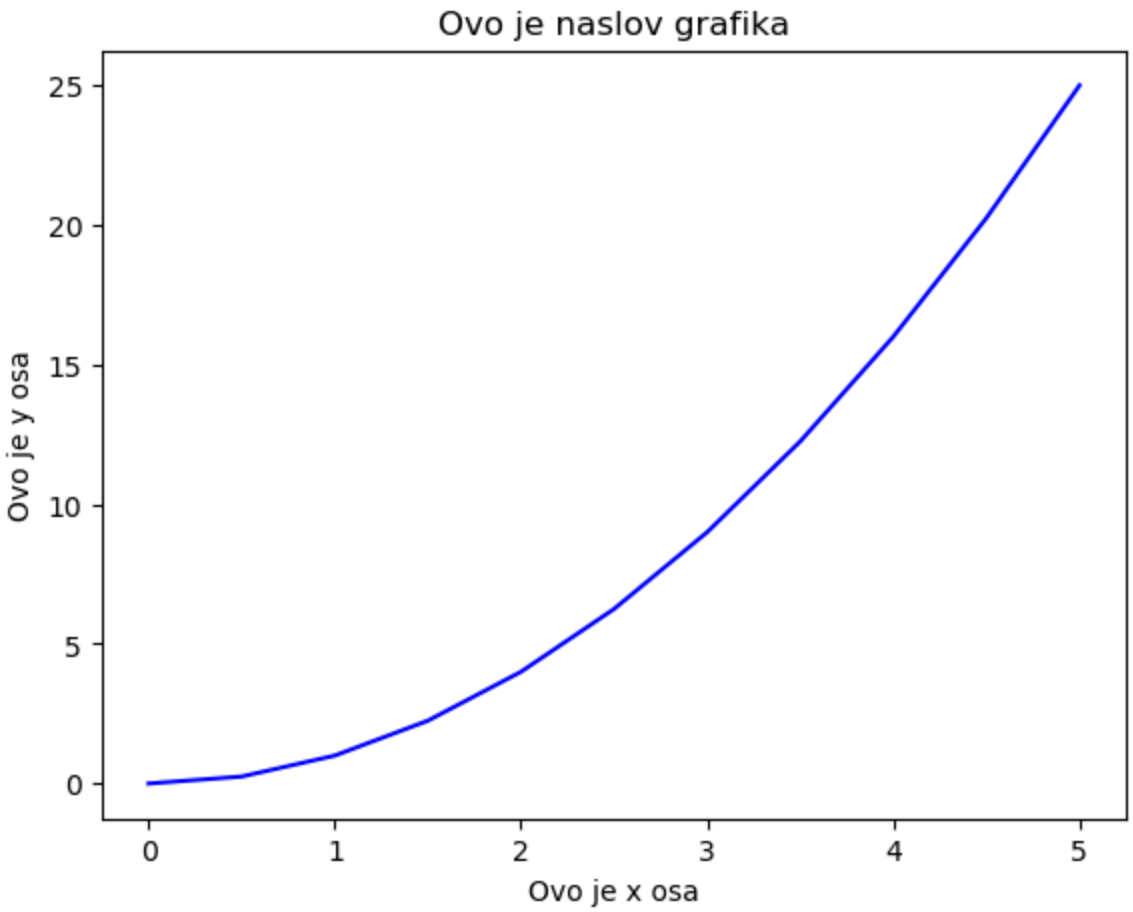


Figure predstavlja samo "platno" na kom iscrtavamo grafik. add_axes funkcija dodaje ose na naš grafik. Parametar prosleđen kao lista respektivno predstavlja procente udaljenosti od leve ivice i od dna cele figure dok sledeća dva parametra ukazuju na širinu i dužinu u procentima cele figure. Kod je naizgled malo kompleksniji nego u ranijim primerima ali nam daje veću fleksibilnost.

`subplots` funkcija će nam pokazati veću automatizovanost kreiranja osa i lakoću kreiranja više grafika odjednom ukoliko je to potrebno.

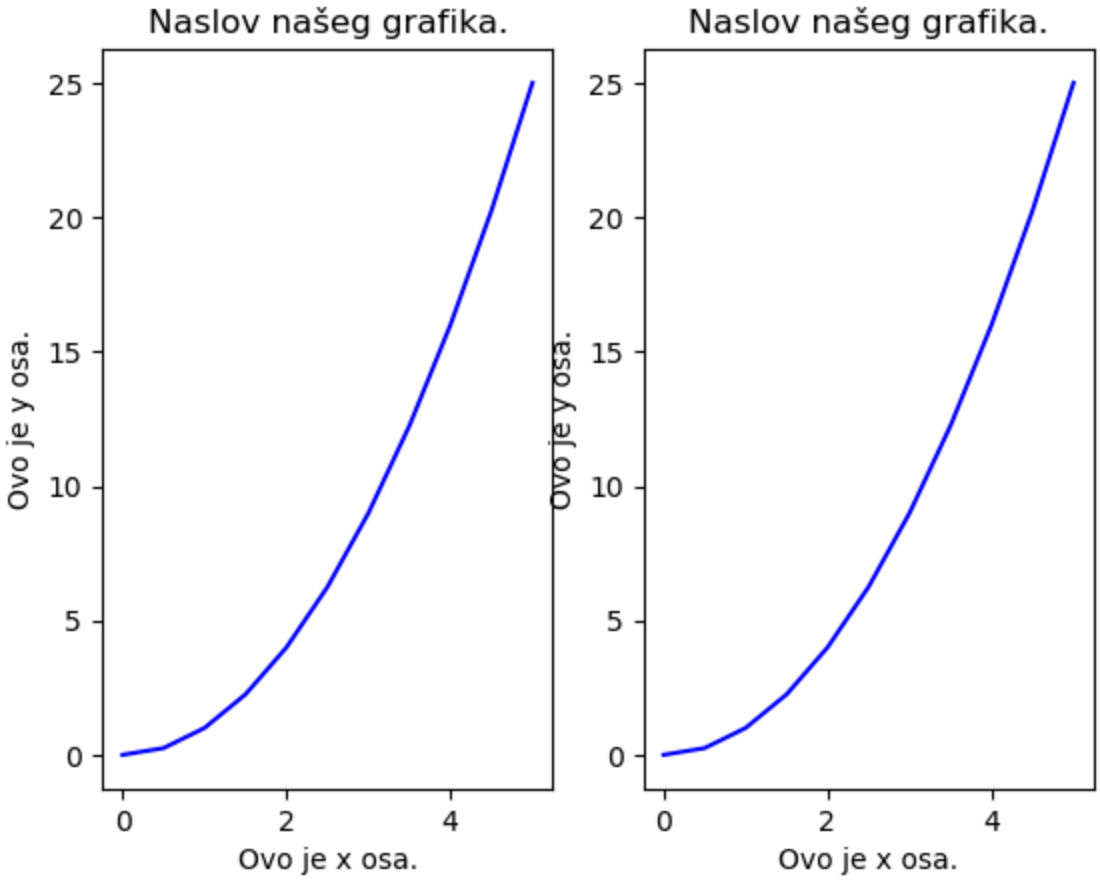
In [7]:
```
fig, axes = plt.subplots() #slično kao plt.figure samo što vraćamo tuple osa i figu
axes.plot(x,y,'r')
axes.set_xlabel('Ovo je x osa.')
axes.set_ylabel('Ovo je y osa.')
axes.set_title('Naslov našeg grafika.')
```

Out[7]:
```
Text(0.5, 1.0, 'Naslov našeg grafika.')
```



Ovako možemo kreirati više grafika i iterirati kroz njih:

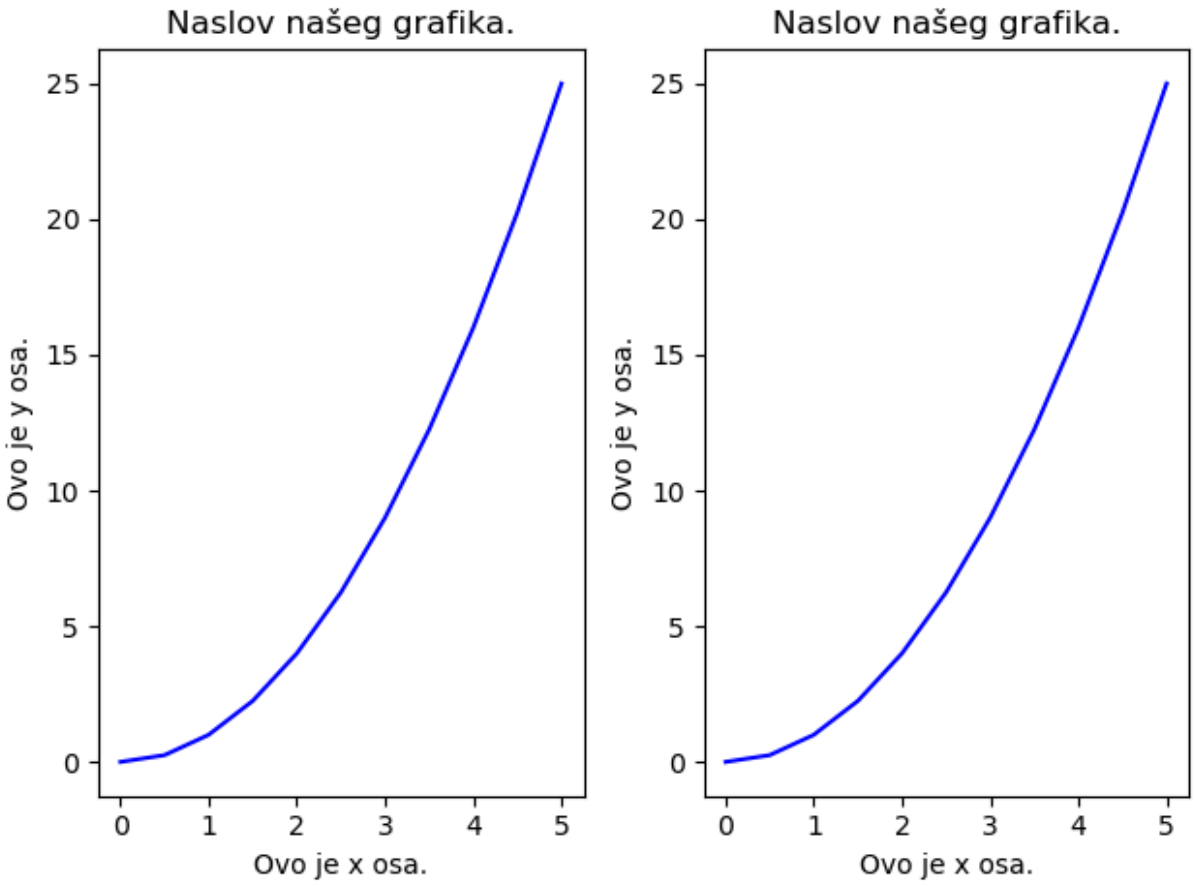
In [8]:
```
fig, axes = plt.subplots(nrows = 1, ncols = 2)
for ax in axes:
    ax.plot(x,y,'b')
    ax.set_xlabel('Ovo je x osa.')
    ax.set_ylabel('Ovo je y osa.')
    ax.set_title('Naslov našeg grafika.')
```



Vidimo da imamo sadržaj koji se preklapa na našem grafiku, stoga možemo pozvati `tight_layout()` da bismo sprečili situaciju da sadržaj jednog grafika prekriva drugi.

In [9]:
```
fig, axes = plt.subplots(nrows = 1, ncols = 2)
for ax in axes:
    ax.plot(x,y,'b')
    ax.set_xlabel('Ovo je x osa.')
    ax.set_ylabel('Ovo je y osa.')
    ax.set_title('Naslov našeg grafika.')

plt.tight_layout()
```



Sa parametrom figsize možemo podešavati širinu i visinu naše figure respektivno:

In [10]:
```
fig = plt.figure(figsize=(8,4))
axes = fig.add_axes([0.1,0.1,0.8,0.8])
axes.plot(x,y,'b')
axes.set_xlabel('Ovo je x osa')
axes.set_ylabel('Ovo je y osa')
axes.set_title('Ovo je naslov grafika')
```

Out[10]:
```
Text(0.5, 1.0, 'Ovo je naslov grafika')
```

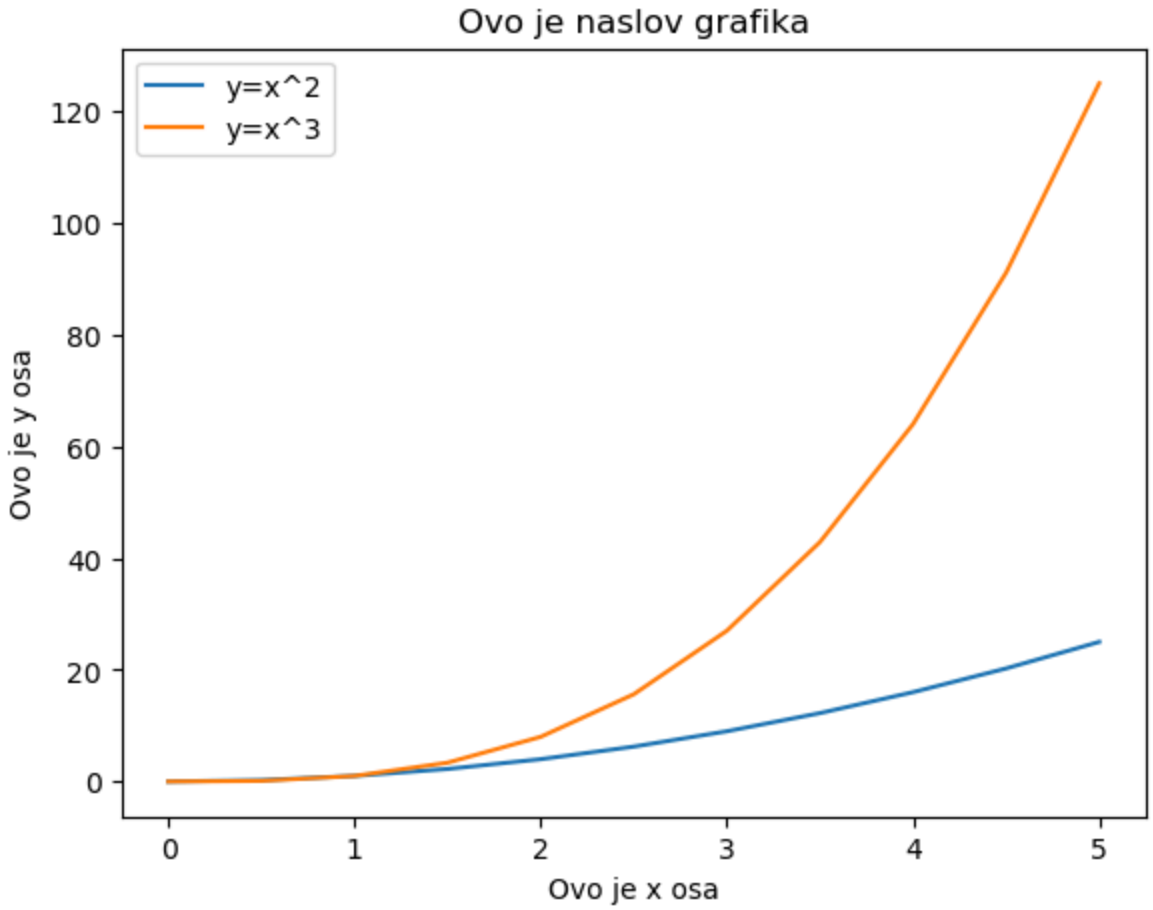


Matplotlib može da čuva svoje grafike u velikom broju formata - PNG, JPG, EPS, SVG, PGF i PDF.

In [11]:
```
fig.savefig("slikaGrafika.png")
```

Videli smo da funkcije set_title, set_xlabel i set_ylabel podešavaju tekst naslova, x i y ose na grafiku ali postoji opcija dodavanja legende na grafik. Pozicija na grafiku se menja prosleđivanjem parametra `loc` u funkciju `legend()`. Najčešće loc = 0 - pušta Matplotlib da odredi nabolju poziciju za legendu.

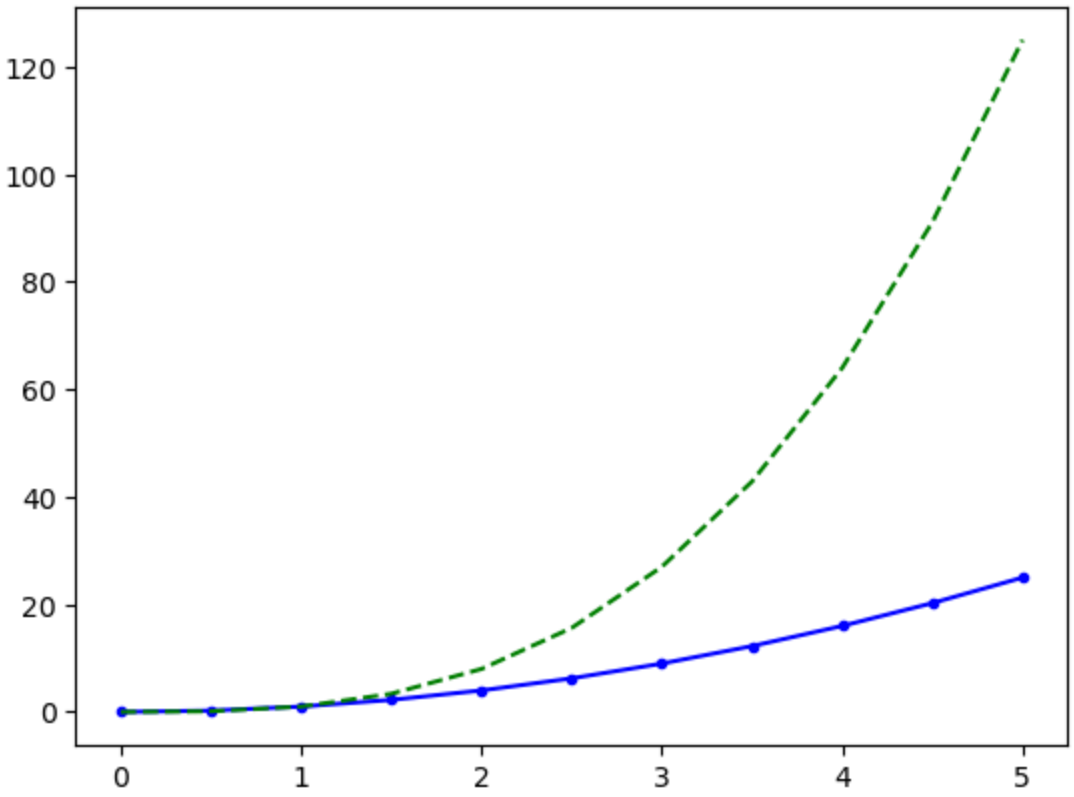
In [12]:
```
fig = plt.figure()
axes = fig.add_axes([0.1,0.1,0.8,0.8])
axes.set_xlabel('Ovo je x osa')
axes.set_ylabel('Ovo je y osa')
axes.set_title('Ovo je naslov grafika')
axes.plot(x,x**2, label="y=x^2")
axes.plot(x,x**3, label = "y=x^3")
axes.legend()
plt.show()
```



Matlab nudi puno opcija za podešavanje tipova linija na graficima, boja linija i slično. Sintaksa je vrlo nalik onoj u Matlab-u. `'b'` je plava boja `'g'` je zelena i tako dalje. Matlab API takođe pruža podršku za izbor stila linije gde je na primer `'b.-'` plava linija sa tačkicama.

In [13]:
```
fig, ax = plt.subplots()
ax.plot(x, x**2, 'b.-')
ax.plot(x, x**3, 'g--')
```

Out[13]:
```
[<matplotlib.lines.Line2D at 0xbd6e7d0>]
```

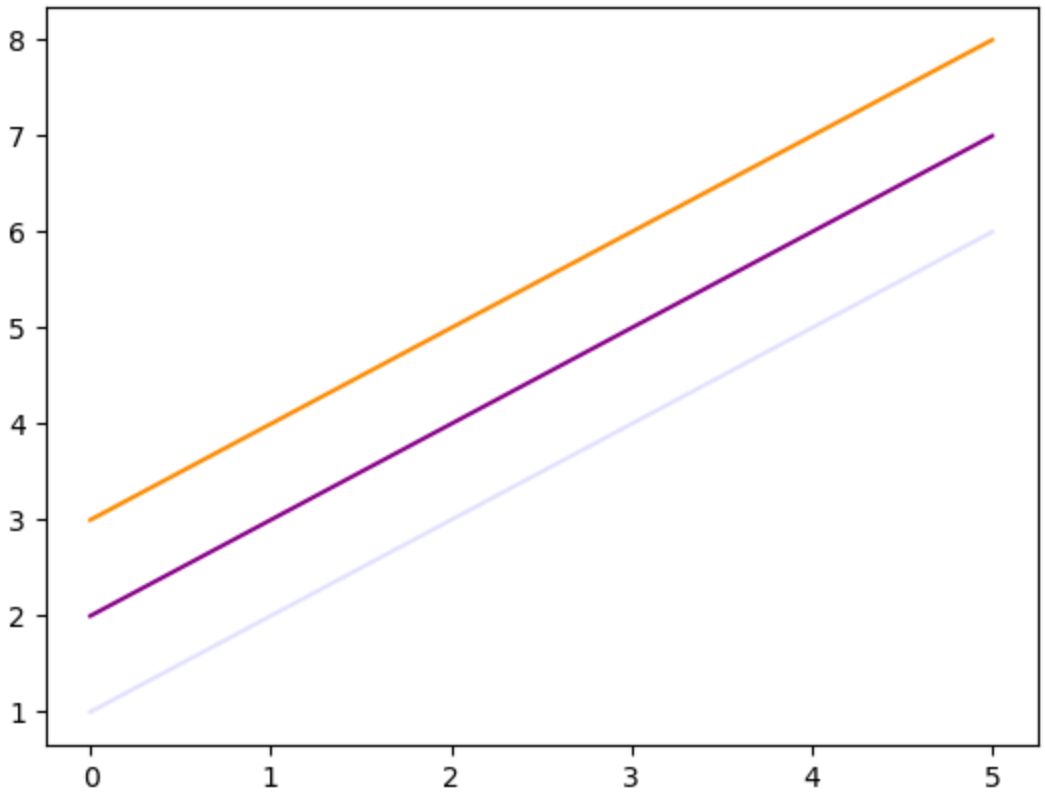


Boja se može podesiti i eksplicitnim dodeljivanjem vrednosti parametro color koji prima heksadecimalne brojeve kao vrednosti boja (ili cele reči) a dodatno se može podesiti i alpha parametar koji reguliše transparentnost.

In [14]:
```
fig, ax = plt.subplots()

ax.plot(x, x+1, color="blue", alpha=0.1)
ax.plot(x, x+2, color="#8B008B")
ax.plot(x, x+3, color="#FF8C00")
```

Out[14]:
```
[<matplotlib.lines.Line2D at 0xc186a10>]
```

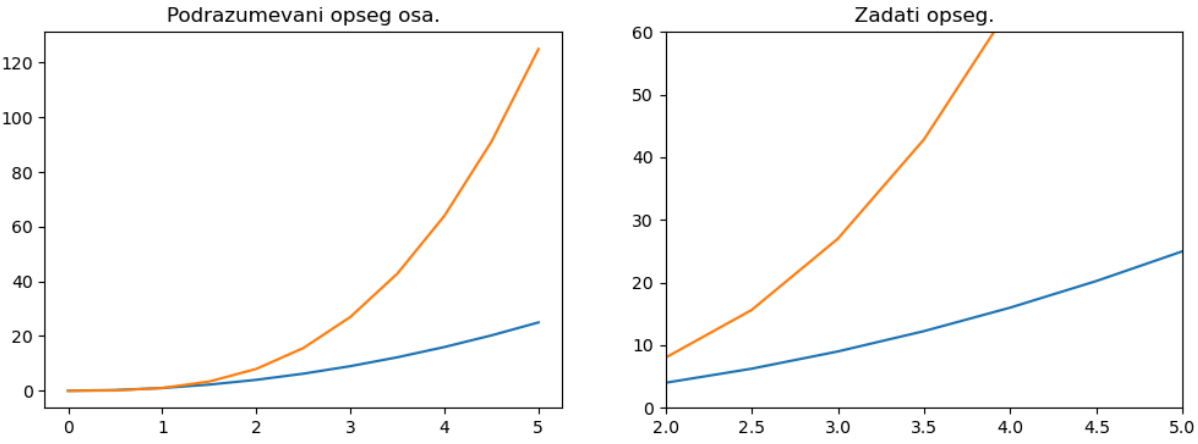


Matplotlib pruža opcije za podešavanje granica u kojima će se kretati vrednosti na y osi uz pomoć funkcije `set_ylim` i x ose uz pomoć funkcije `set_xlim()`.

In [15]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(x, x**2, x, x**3)
axes[0].set_title("Podrazumevani opseg osa.")

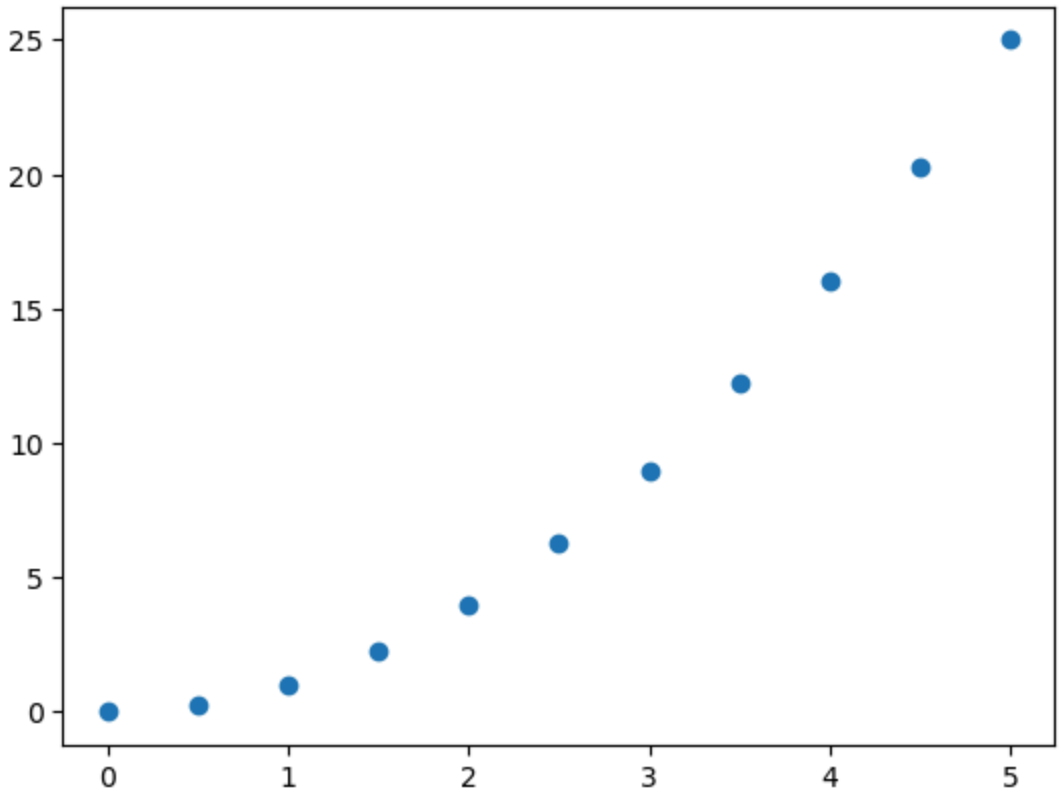

axes[1].plot(x, x**2, x, x**3)
axes[1].set_ylim([0, 60])
axes[1].set_xlim([2, 5])
axes[1].set_title("Zadati opseg.");
```



Postoje naravno i druge vrste grafika poput scatterplot-ova, histograma ili boxplot-ova. Demonstrirajmo njihov izgled.

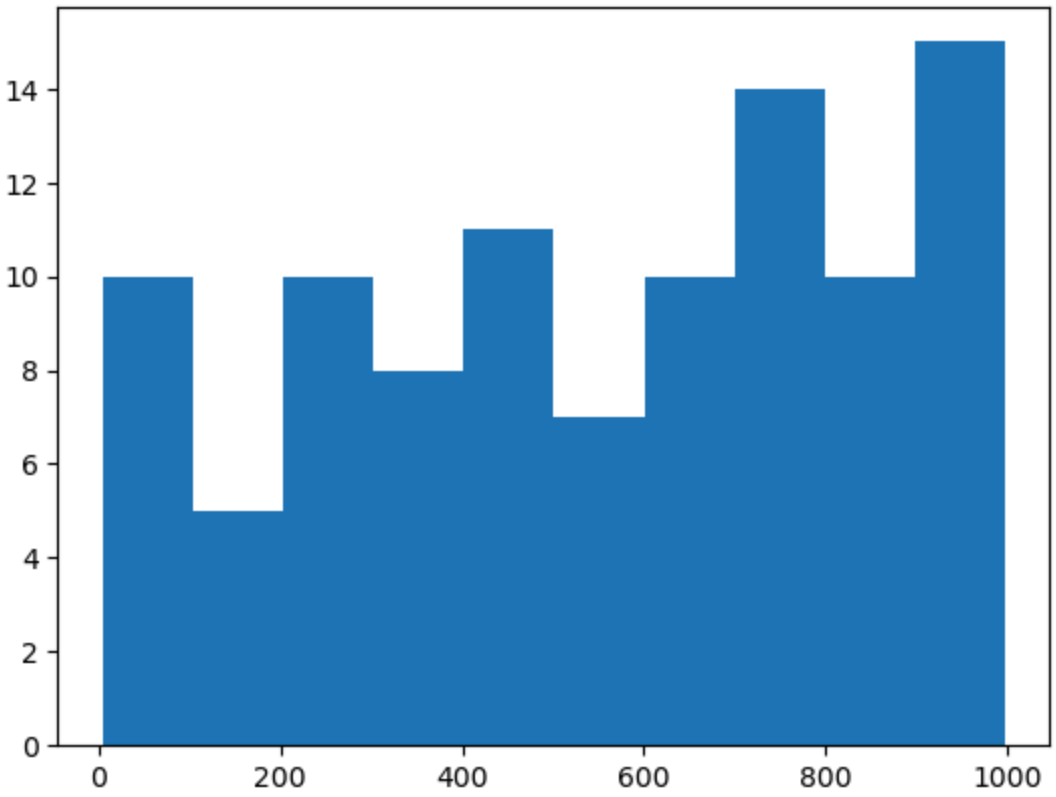
In [16]:
```
plt.scatter(x,y)
```

Out[16]:
```
<matplotlib.collections.PathCollection at 0xc1ba810>
```



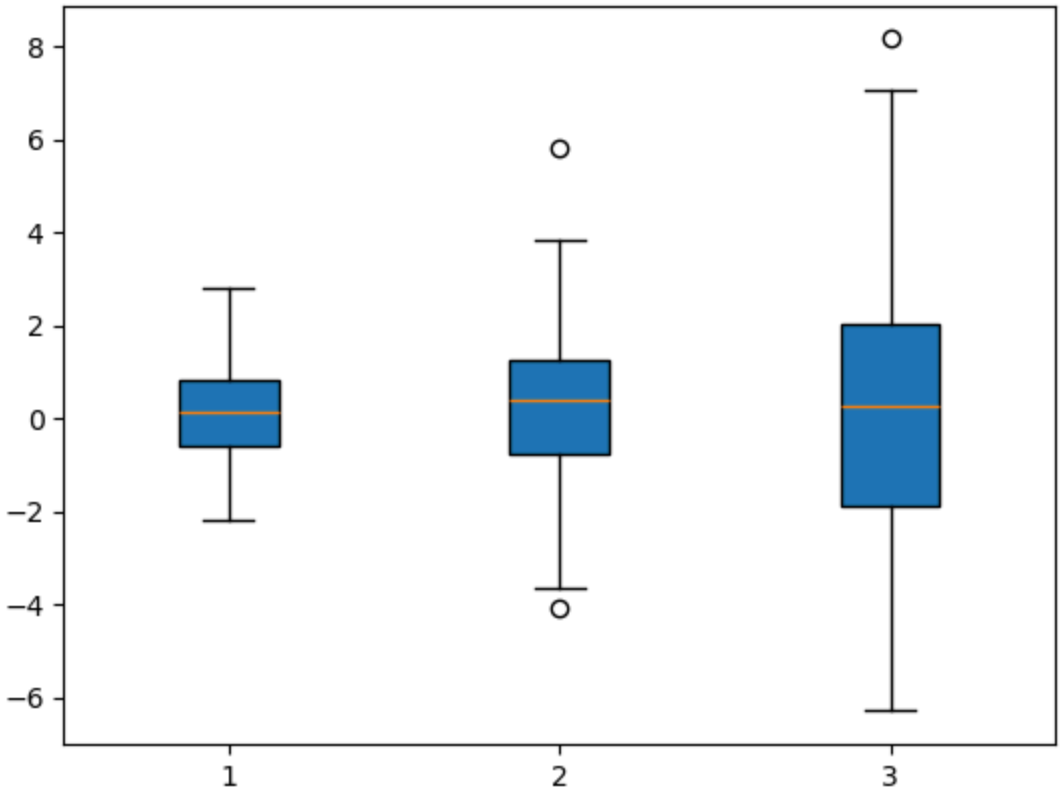
In [17]:
```
from random import sample
data = sample(range(1, 1000), 100)
plt.hist(data)
```

Out[17]:
```
(array([10.,  5., 10.,  8., 11.,  7., 10., 14., 10., 15.]),
 array([  5. , 104.2, 203.4, 302.6, 401.8, 501. , 600.2, 699.4, 798.6,
        897.8, 997. ]),
 <BarContainer object of 10 artists>)
```



In [18]:
```
data = [np.random.normal(0, std, 100) for std in range(1, 4)]

# rectangular box plot
plt.boxplot(data,vert=True,patch_artist=True);
```



Sekcija 1 - Matplotlib

Seaborn je još jedna biblioteka pogodna za vizualizaciju podataka sa kojom možemo iscrtavati i kompleksnije vrste grafikona pa je veoma popularna i korišćena. Prikažimo rad ove biblioteke tako što ćemo početi od grafikona koji dobro vizualizuju raspodele promenljivih. Za sada ćemo koristiti dataset koji Seaborn biblioteka sadrži.

In [19]:
```
import seaborn as sns
```

In [20]:
```
tips = sns.load_dataset('tips')
```

In [21]:
```
tips.head() #setimo se Pandas biblioteke - želimo da pogledamo strukturu dataframe-
```

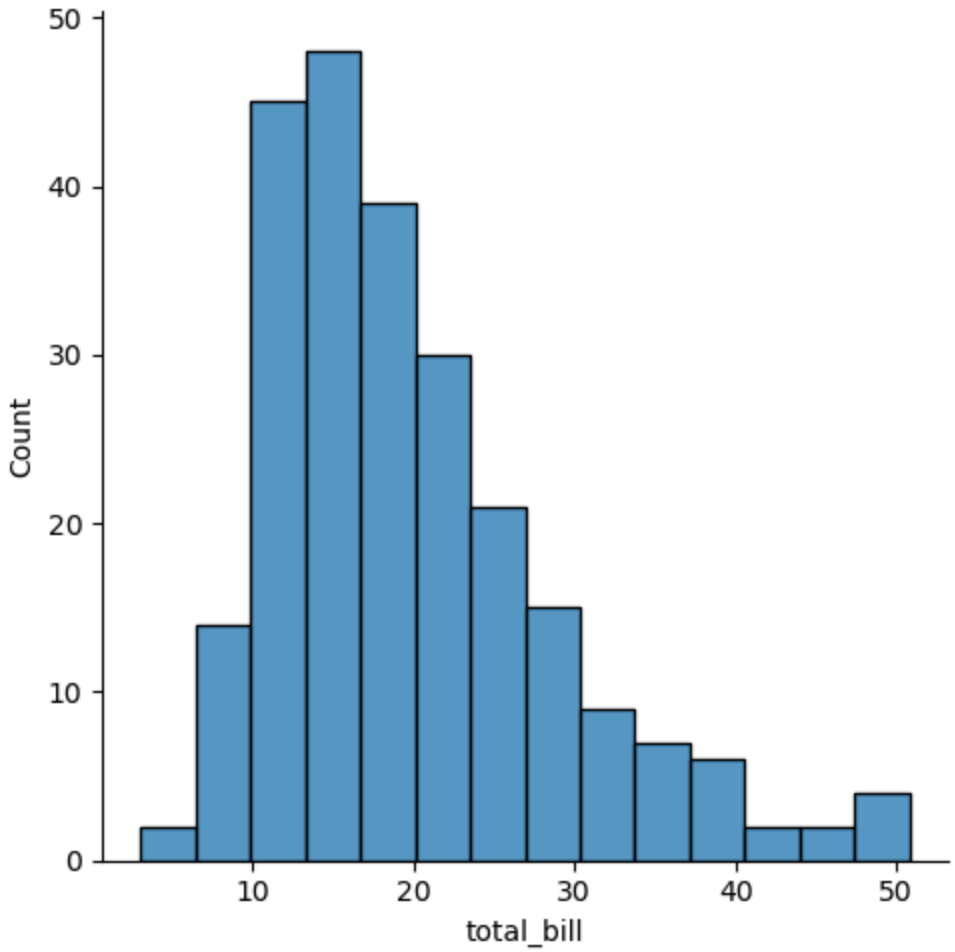
Out[21]:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

Funkcija `distplot` nam služi za vizualizaciju raspodele jedne promenljive - recimo da želimo da pogledamo raspodelu ukupne vrednosti računa:

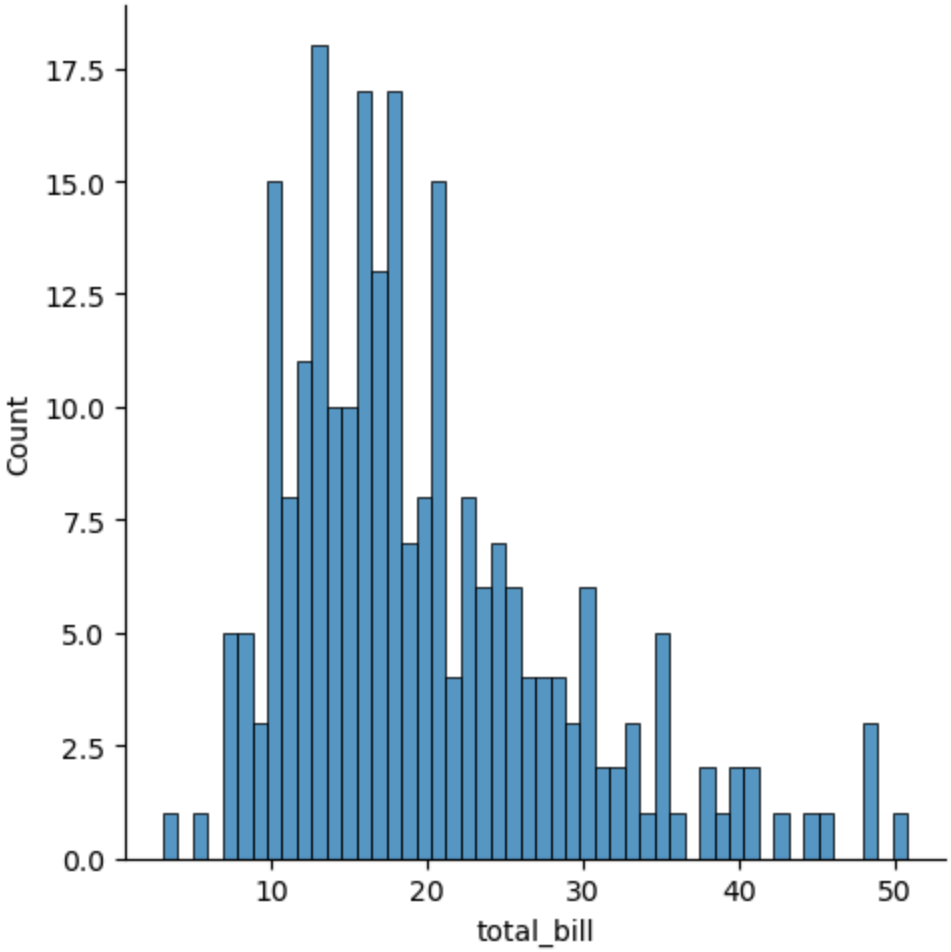
In [22]:
```
sns.displot(tips['total_bill'])
```

Out[22]:
```
<seaborn.axisgrid.FacetGrid at 0x1061b0d0>
```



In [23]:
```
sns.displot(tips['total_bill'],kde=False,bins=50)
```

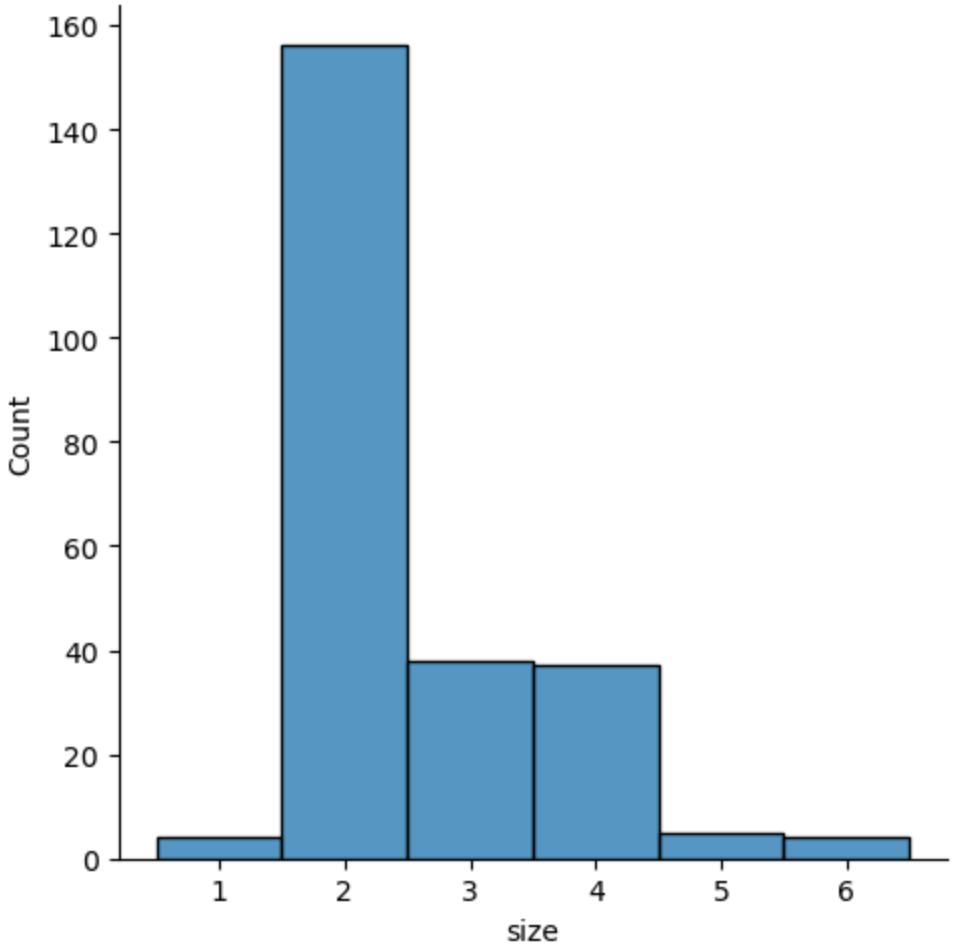
Out[23]:
```
<seaborn.axisgrid.FacetGrid at 0x119e05d0>
```



Primetimo razliku u specifikaciji broja 'bin'-ova koje smo prosledili funkciji. Ukoliko sa ovom funkcijom želimo da vizualizujemo diskretne numeričke podatke to možemo postići prosleđivanjem parametra discrete = true.

In [24]:
```
sns.displot(tips['size'],discrete=True)
```

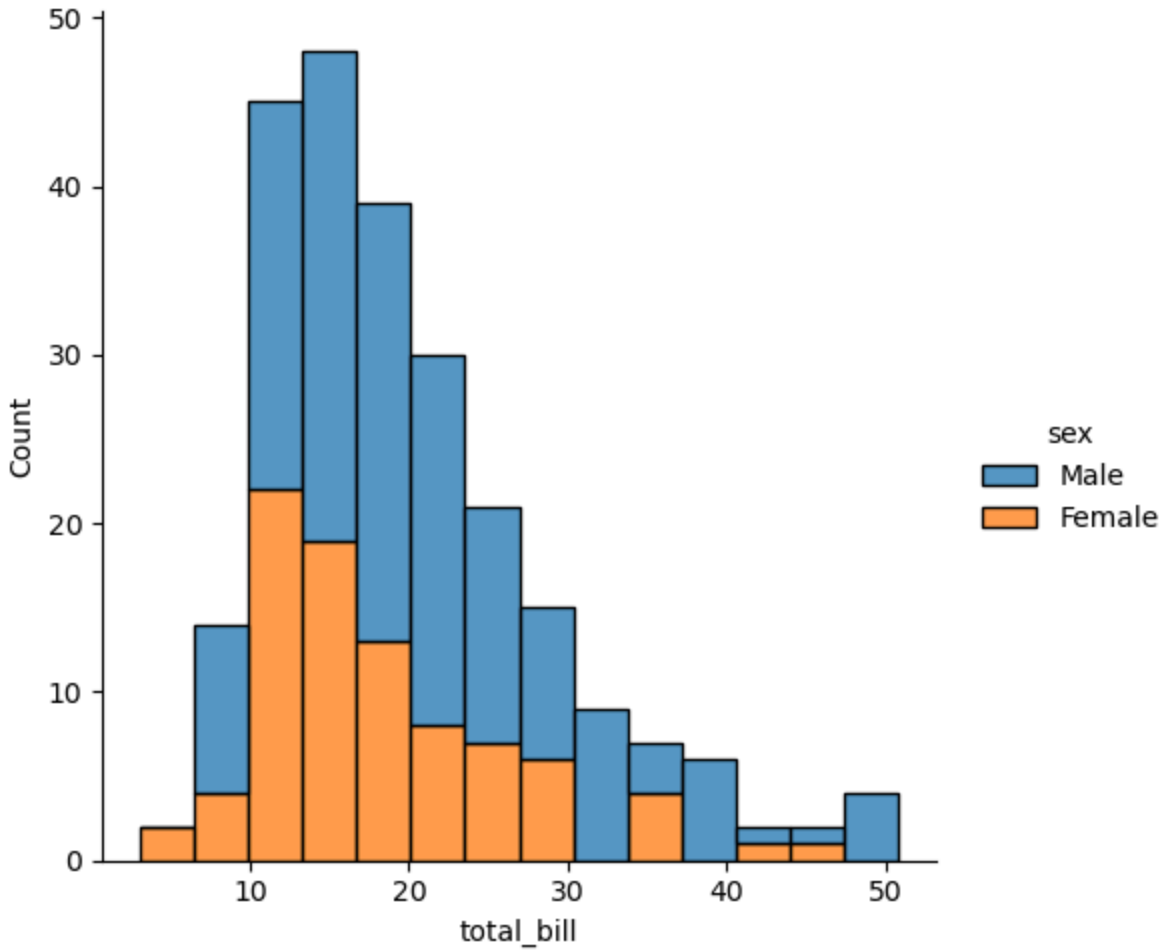
Out[24]:
```
<seaborn.axisgrid.FacetGrid at 0x119e2b10>
```



Još jedan zanimljiv vid prikaza raspodele je prikaz po nekim kategorijama. Recimo da želimo da vidimo raspodelu ukupnog iznosa na računu u zavosnosti od pola.

In [25]:
```
sns.displot(tips,x = 'total_bill',hue = 'sex',multiple='stack')
```

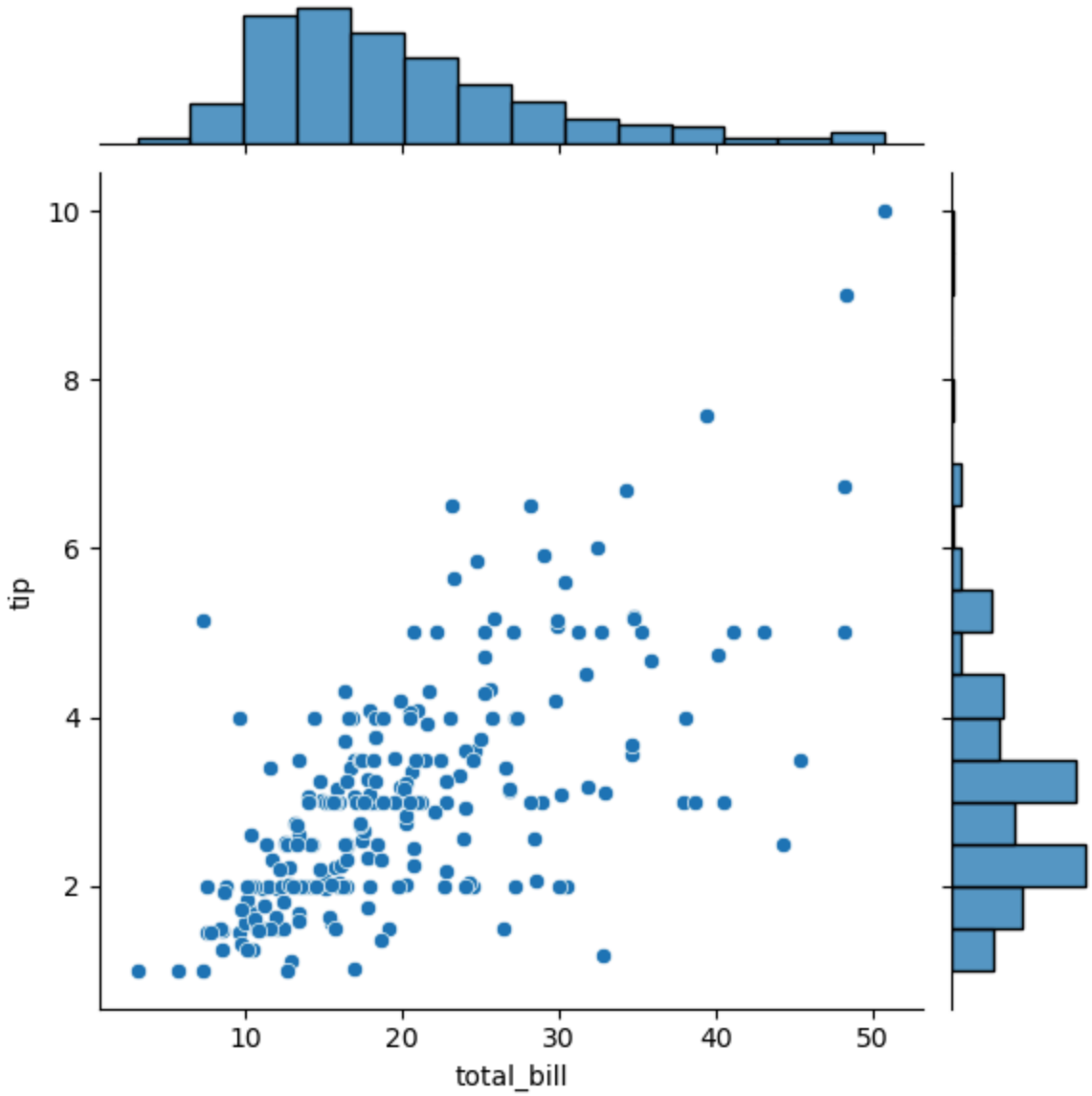
Out[25]:
```
<seaborn.axisgrid.FacetGrid at 0x11aa30d0>
```



Jointplot nam zapravo daje vizualizaciju po dve promenljive gde jedna uzima vrednosti na x osi a druga na y osi.

In [26]:
```
sns.jointplot(x='total_bill',y='tip',data=tips,kind='scatter')
```

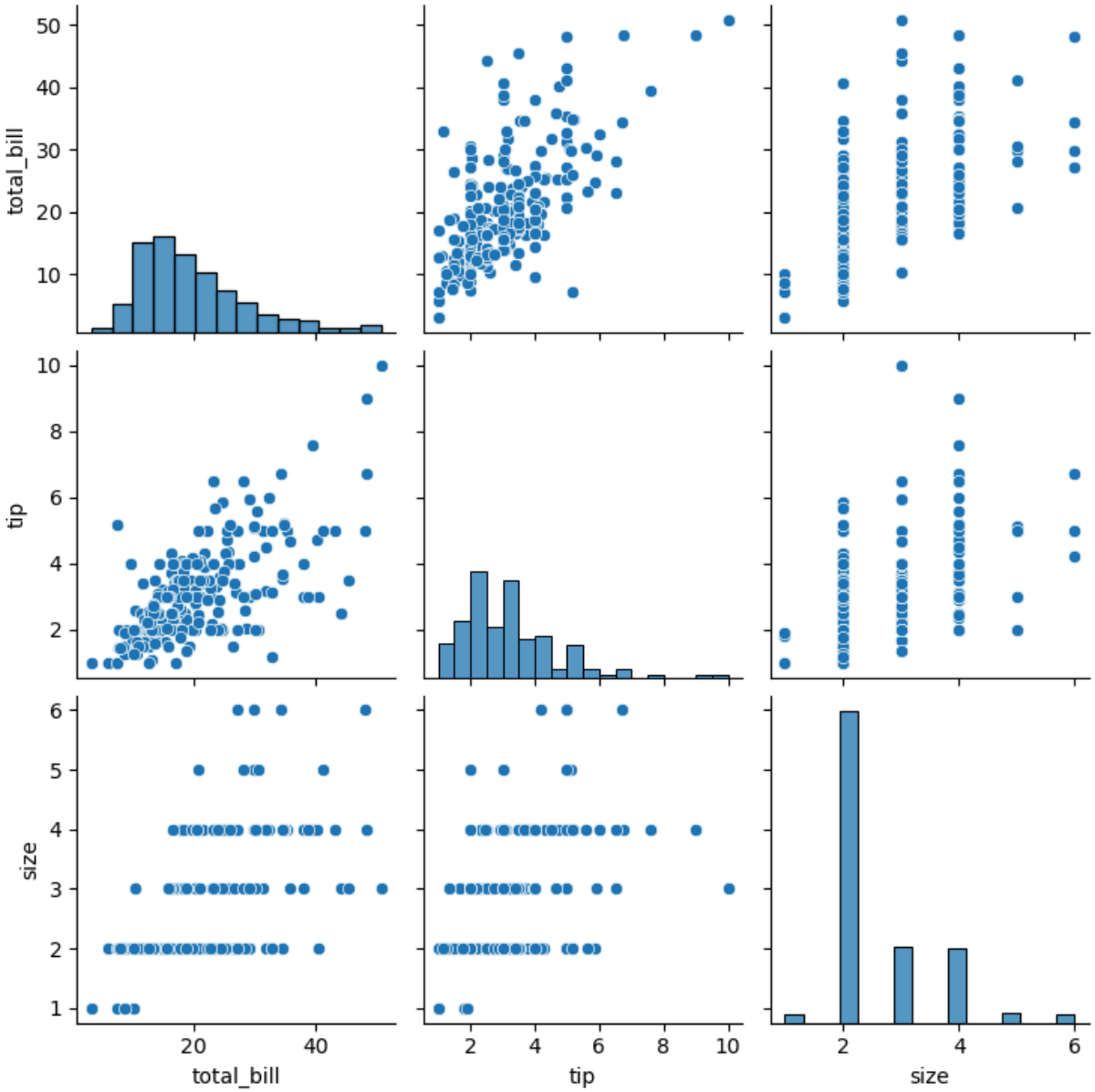
Out[26]:
```
<seaborn.axisgrid.JointGrid at 0x119a99d0>
```



Vidimo da postoji trend veći račun - veća napojnica, međutim postoji i dosta ekstremnih vrednosti, ne možemo uočiti neku perfektnu korelaciju. Pairplot je još jedan od grafika koje možemo iskoristiti - kreiraće grafik za svaki od parova kolona, potencijalno pruživši informacije o nekoj korelisanosti u podacima.

In [27]:
```
sns.pairplot(tips)
```

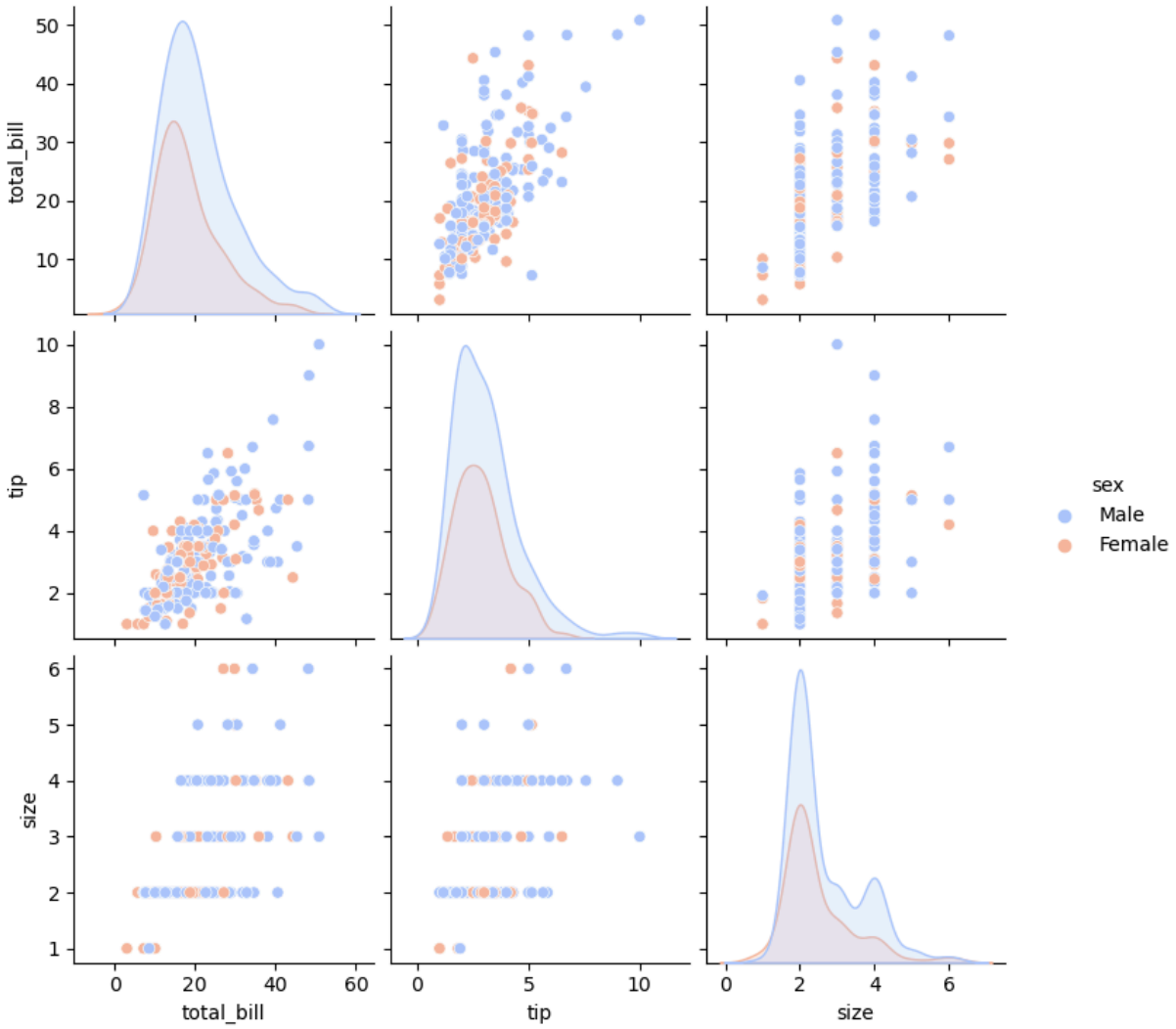
Out[27]:
```
<seaborn.axisgrid.PairGrid at 0x11bce810>
```



I ovde možemo zahtevati od Seaborn-a da naše podatke iscrta tako što će podeliti podatke po kategoriji - u ovom slučaju po polu.

In [28]:
```
sns.pairplot(tips,hue='sex',palette='coolwarm')
```

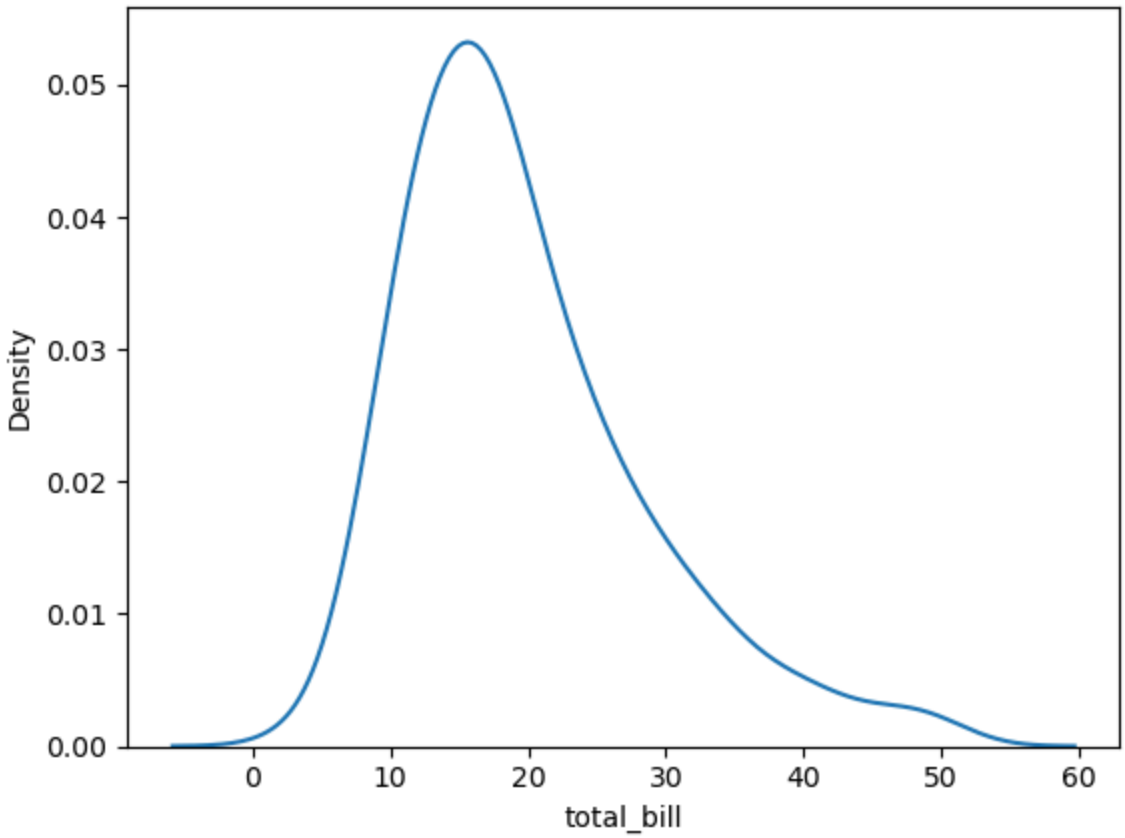
Out[28]:
```
<seaborn.axisgrid.PairGrid at 0x11d2d110>
```



Postoji još jedan značajan vid grafičkog prikazivanja raspodele neke numeričke vrednosti - KDE plot. KDE je skraćenica za `Kernel Density Estimation` što je metoda za procenu raspodele numeričke vrednosti. KDE grafik suštinski menja svaku observaciju u skupu podataka sa normalnom raspodelom koja je centrirana oko te observacije i računa zbir tako dobijenih normalnih raspodela.

In [29]:
```
sns.kdeplot(tips['total_bill'])
```

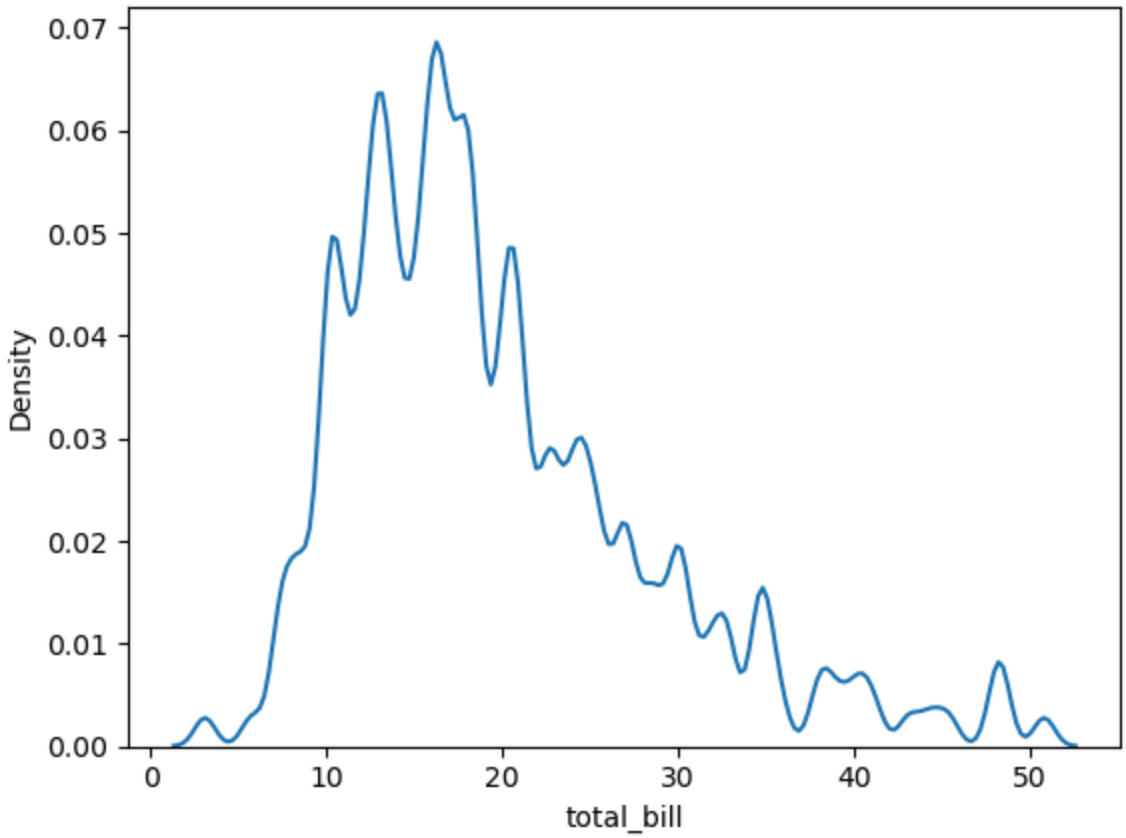
Out[29]:
```
<Axes: xlabel='total_bill', ylabel='Density'>
```



Parametri koje možemo proslediti KDE grafiku su parametar za `smoothing` i parametar koji će funkciju gustine zameniti sa funkcijom raspodele. Prikažimo smoothing na niskom nivou - povećavamo varijansu raspodele sa puno pikova i visok nivo smoothing-a, gde postoji rizik od eliminisanja ekstremnih vrednosti.

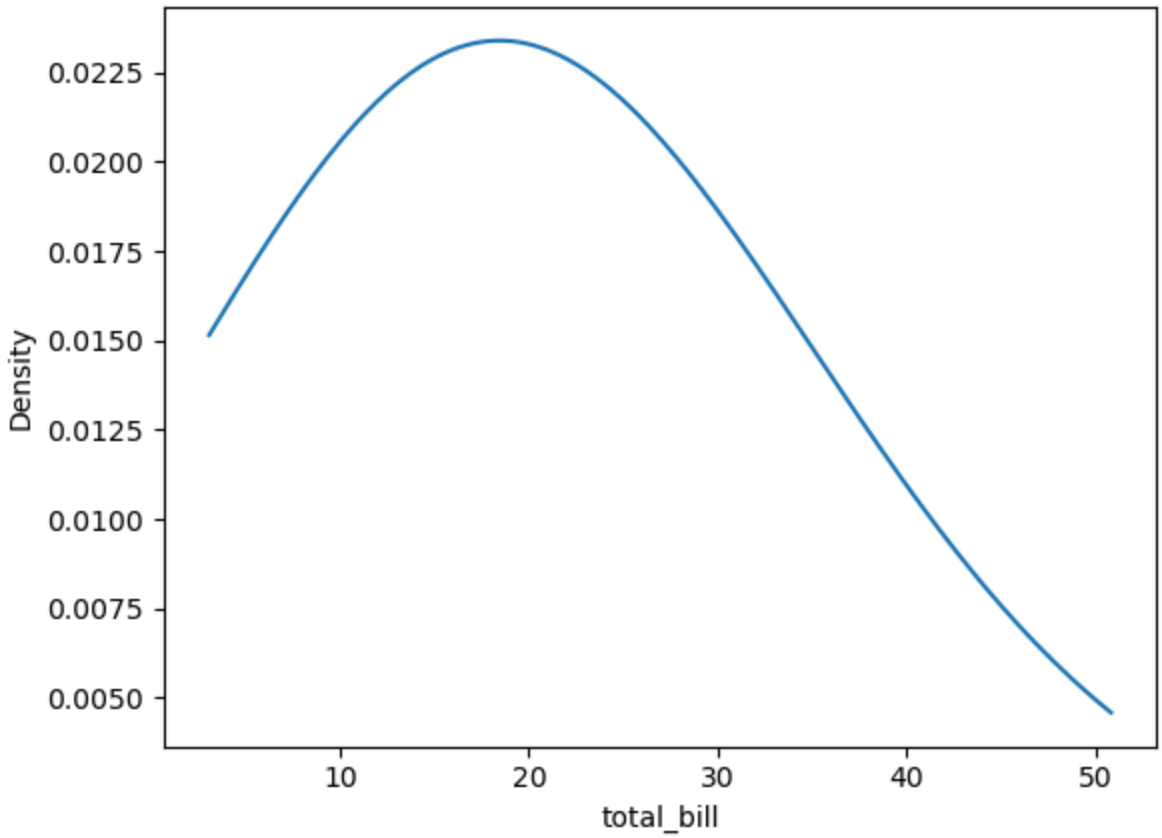
In [30]:
```
sns.kdeplot(data=tips, x="total_bill", bw_adjust=.2)
```

Out[30]:
```
<Axes: xlabel='total_bill', ylabel='Density'>
```



In [32]:
```
sns.kdeplot(data=tips, x = "total_bill",bw_adjust=5,cut=0)
```

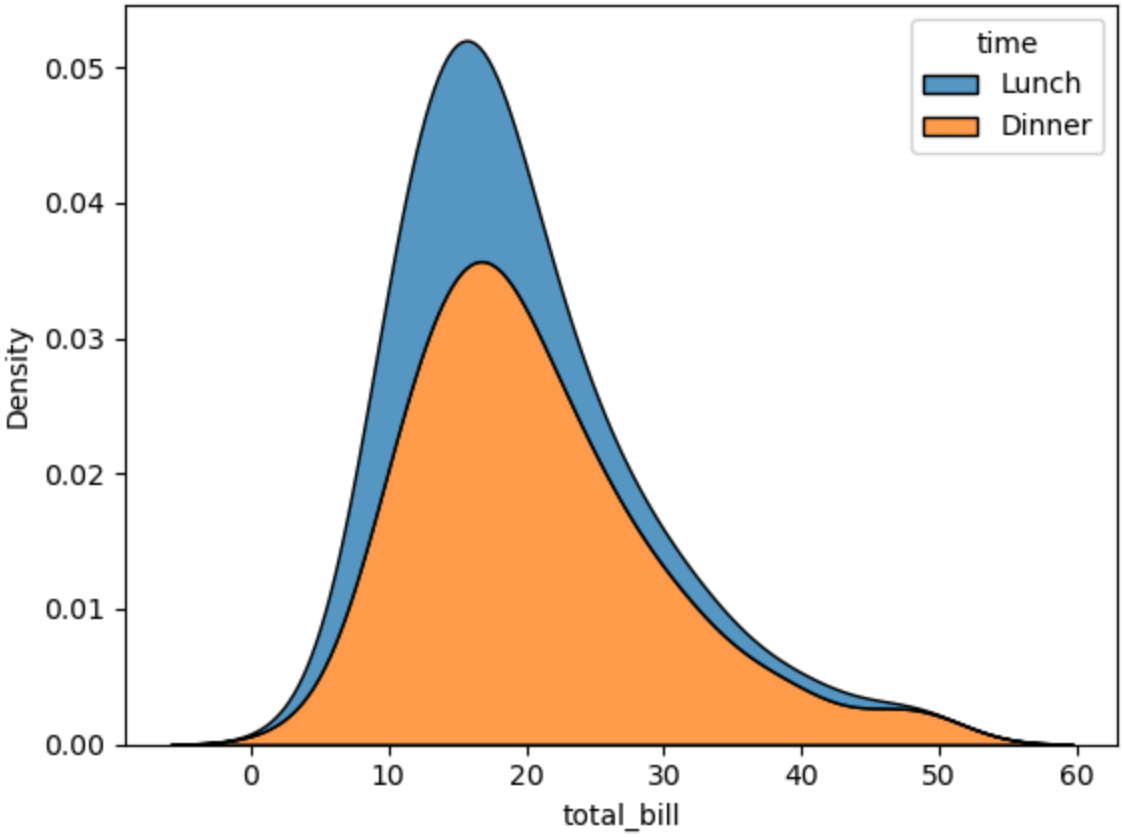
Out[32]: <Axes: xlabel='total_bill', ylabel='Density'>



Takođe možemo razdvajati po nekoj kategoriji, recimo vreme:

In [33]:
```
sns.kdeplot(data=tips, x="total_bill", hue="time", multiple="stack")
```

Out[33]:
```
<Axes: xlabel='total_bill', ylabel='Density'>
```

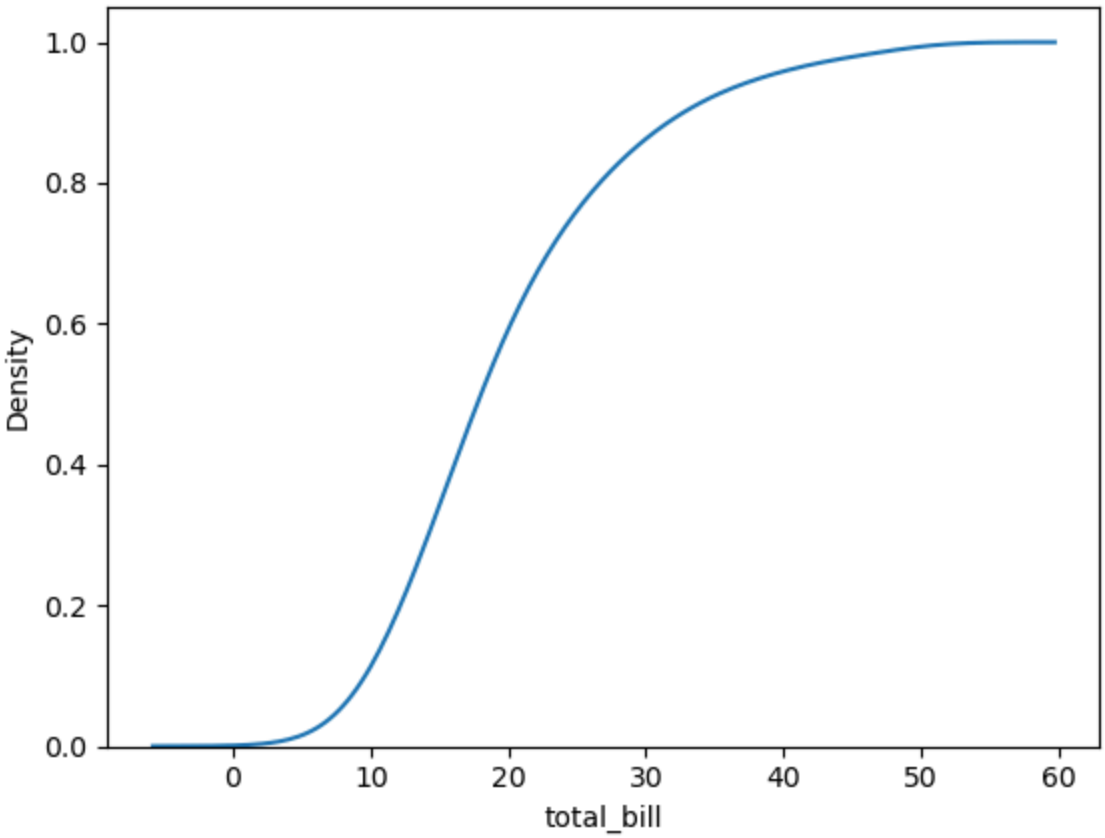


Na ovaj način dobijamo funkciju raspodele, prosleđivanjem parametra `cumulative = True`.

In [34]:
```
import scipy
```

In [35]:
```
sns.kdeplot(tips["total_bill"],cumulative = True)
```

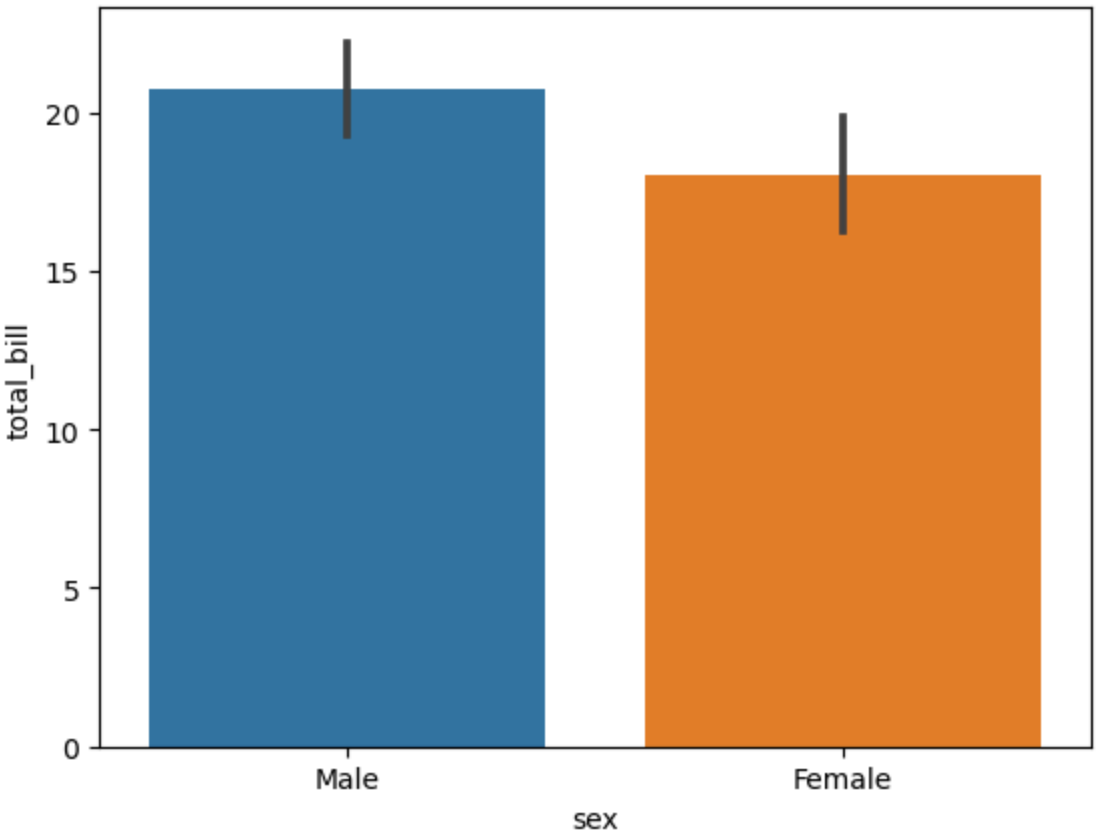
Out[35]:
```
<Axes: xlabel='total_bill', ylabel='Density'>
```



Pokažimo još načina da vizualizujemo podatke delivši ih po nekoj kategoriji. Recimo da želimo da grafički prikažemo srednje vrednosti iznosa računa za žene i muškarce i pogledamo koji pol u proseku izdvaja više novca za račune. Ovakav prikaz možemo ostvariti korišćenjem `barplot` grafikona.

In [36]:
```
sns.barplot(x = 'sex', y = 'total_bill', data = tips)
```

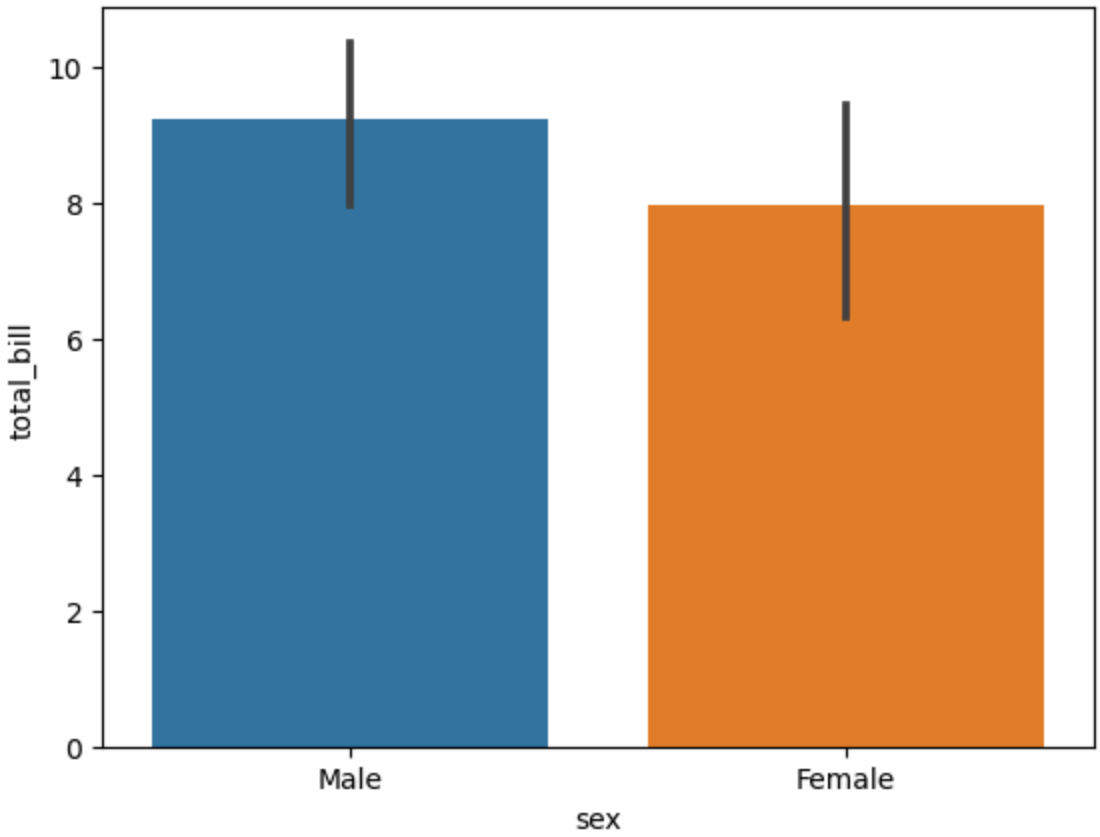
Out[36]:
```
<Axes: xlabel='sex', ylabel='total_bill'>
```



Vidimo da muškarci u proseku izdvajaju više novca za račune u restoranima. Naravno barplot-u možemo proslediti i drugu vrstu informacije oko koje će se grupisati kategorije, recimo standardnu devijaciju.

In [37]:
```
import numpy as np
sns.barplot(x='sex',y='total_bill',data=tips,estimator=np.std)
```

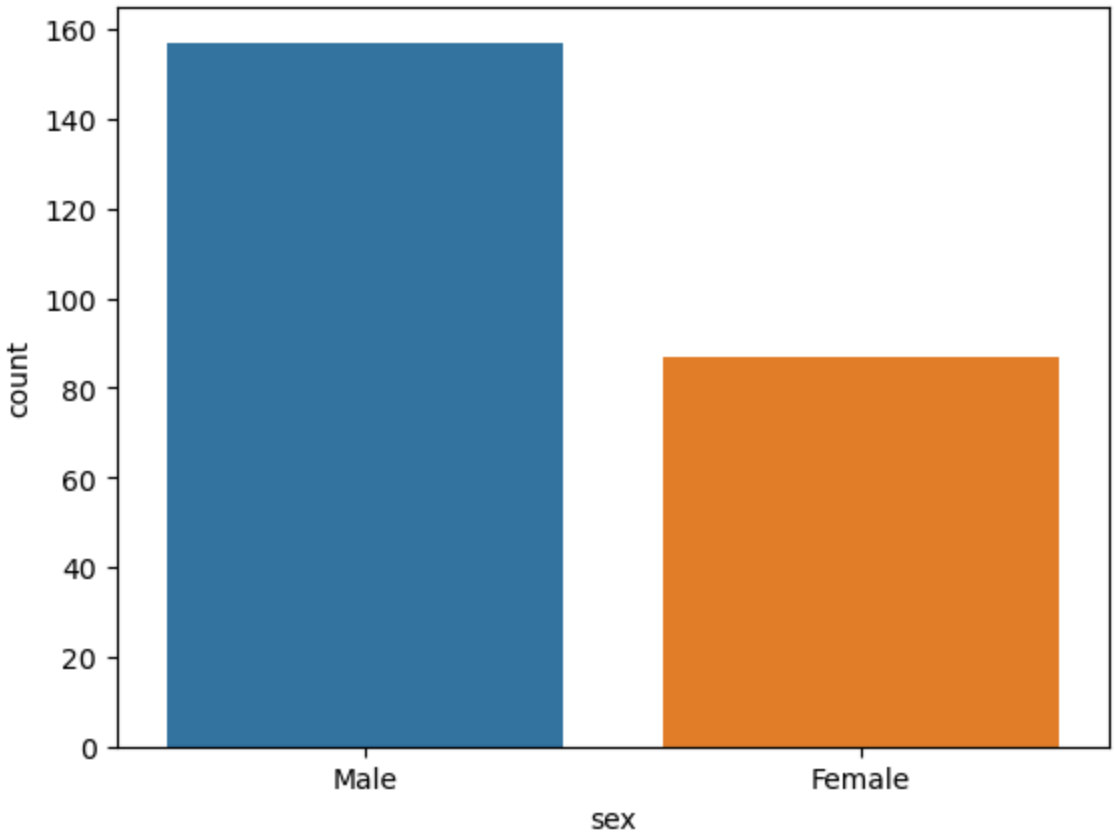
Out[37]:
```
<Axes: xlabel='sex', ylabel='total_bill'>
```



Takođe vidimo da je kod muškaraca izraženo i veće odstupanje od prosečne vrednosti cene računa. Još jedan grafik koji je esencijalno veoma sličan barplot-u samo što grupiše podatke i računa broj opservacija po kategoriji je `countplot`.

In [38]:
```
sns.countplot(x='sex',data=tips)
```

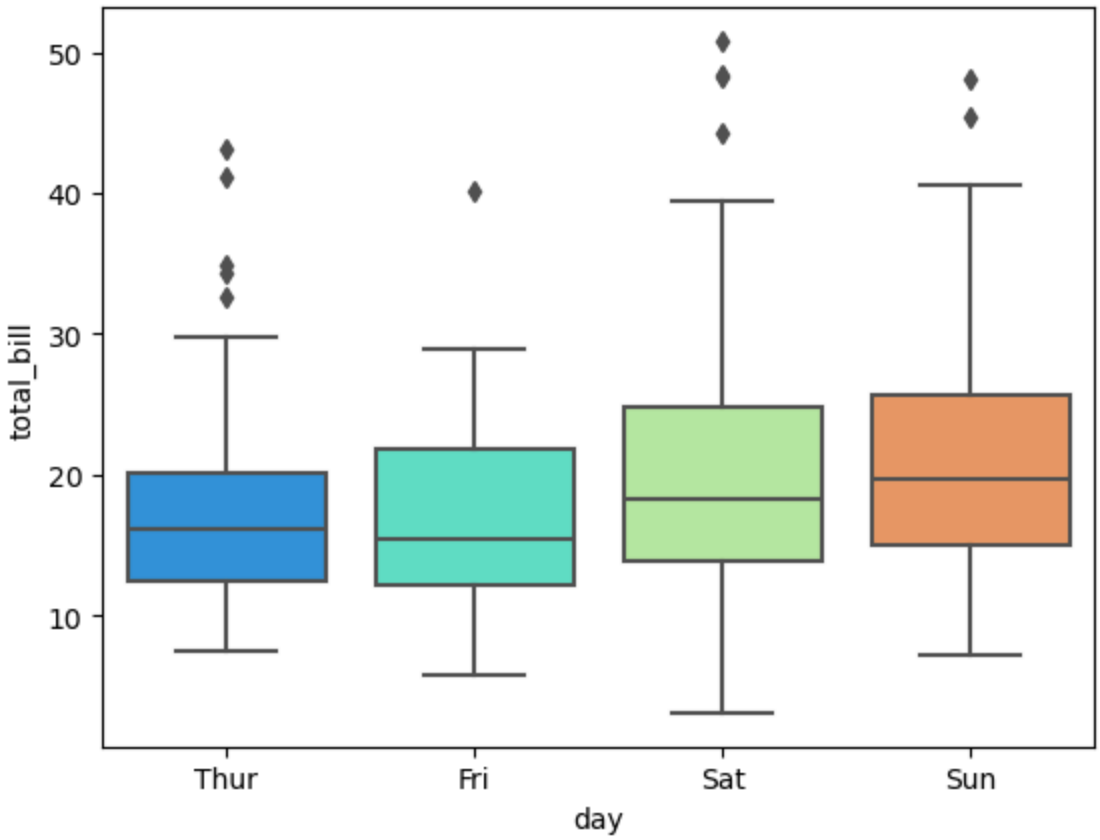
Out[38]:
```
<Axes: xlabel='sex', ylabel='count'>
```



Već smo pokazali boxplot kao jedan od informativnijih i korisnijih grafikona ali njega takođe možemo iskoristiti da pogledamo razlike u raspodeli neke promenljive po kategoriji i identifikujemo potencijalne autlajere u podacima.

In [40]:
```
sns.boxplot(x="day", y="total_bill", data=tips, palette = 'rainbow')
```

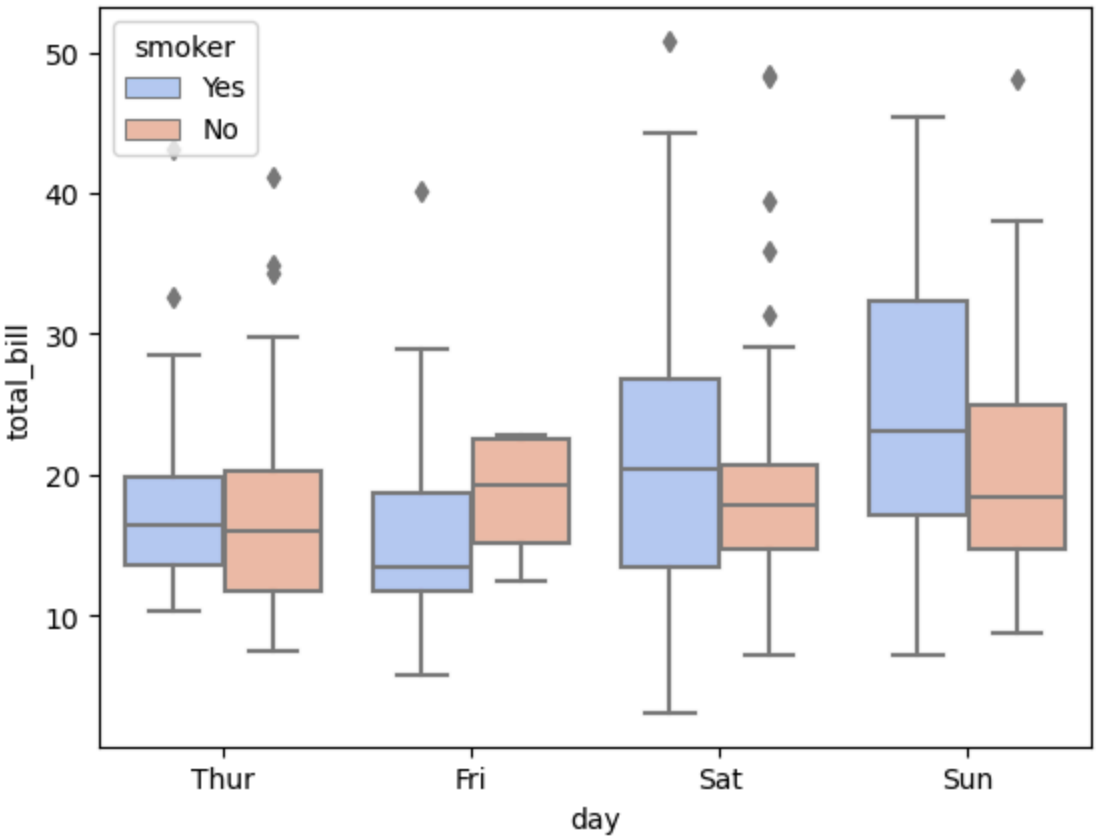
Out[40]:
```
<Axes: xlabel='day', ylabel='total_bill'>
```



Možemo dalje produbiti potragu za nekim pravilima u podacima. Recimo da ćemo totalni iznos na računu da podelimo po danima i po tome da li je osoba pušač ili ne:

In [41]:
```
sns.boxplot(x="day",y="total_bill", data=tips, hue = "smoker", palette="coolwarm")
```

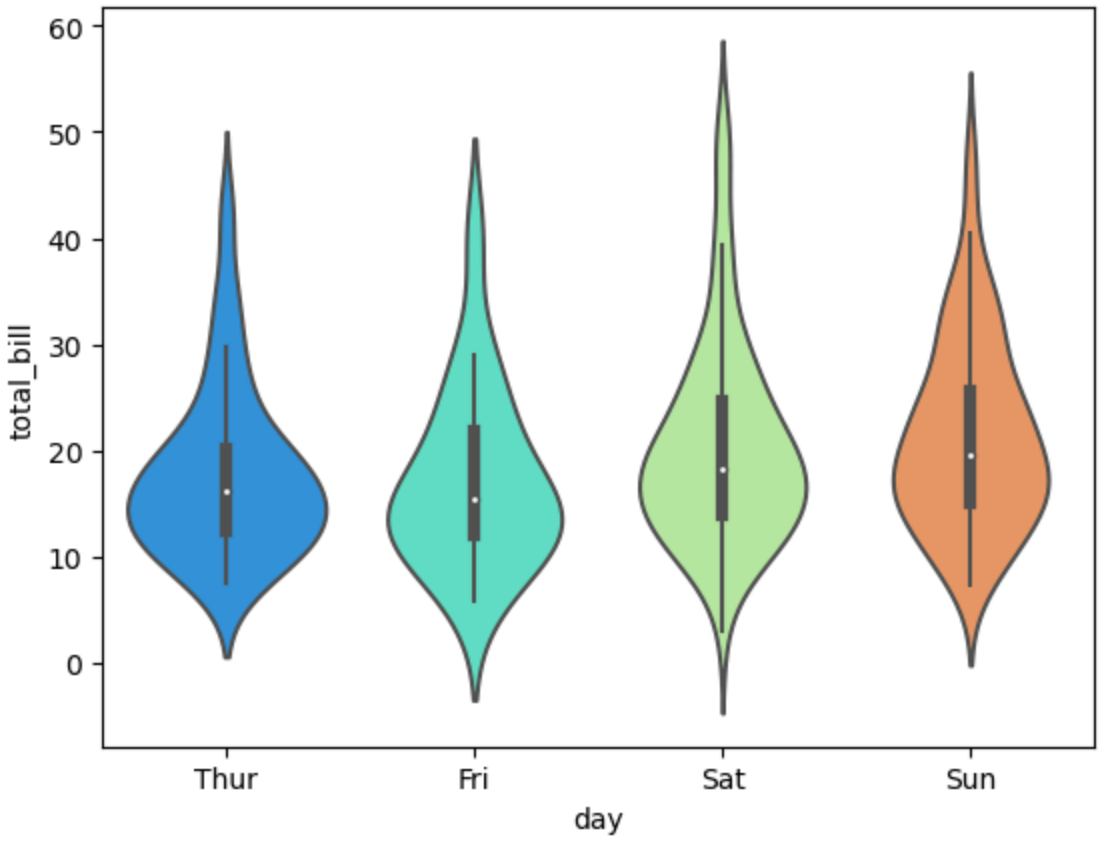
Out[41]:
```
<Axes: xlabel='day', ylabel='total_bill'>
```



Grafik jako sličan boxplot-u je `violinplot` gde se u stvari koristi KDE pristup za procenu navedene raspodele dok se kod boxplot-a pojavljuju konkretne observacije.

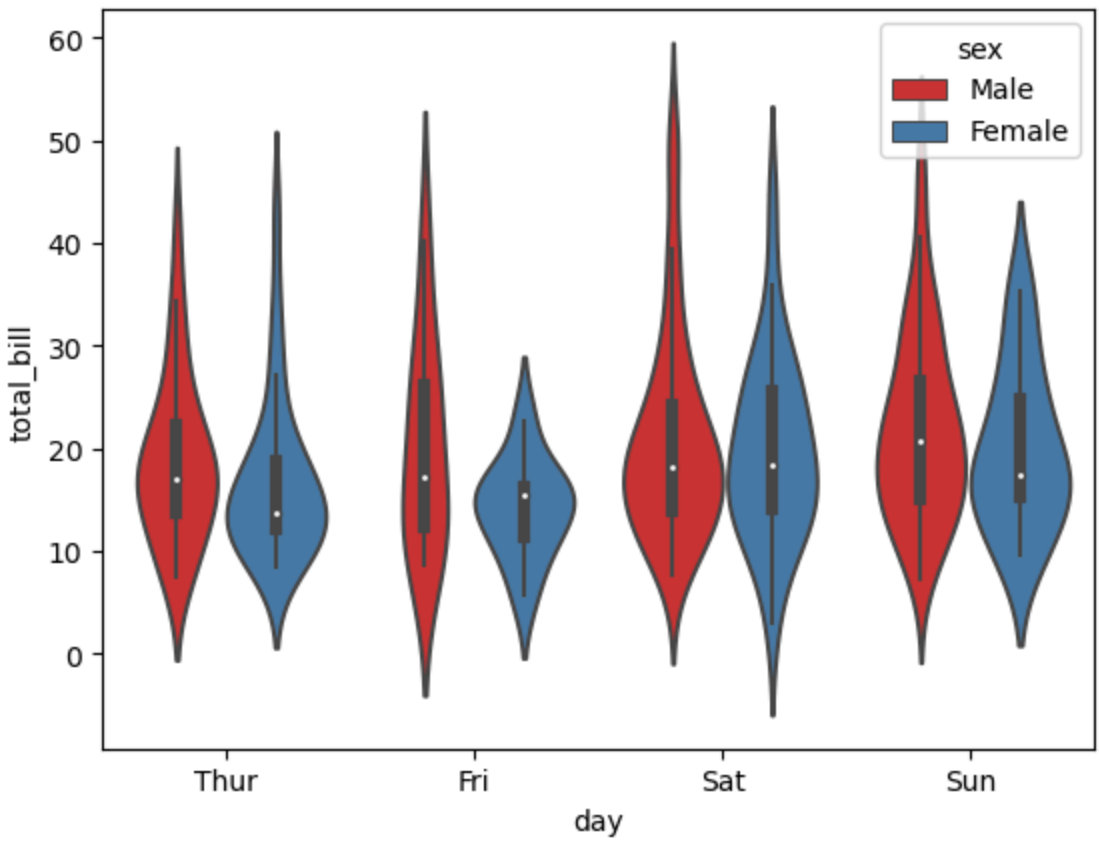
In [42]:
```
sns.violinplot(x="day", y="total_bill", data=tips,palette='rainbow')
```

Out[42]:
```
<Axes: xlabel='day', ylabel='total_bill'>
```



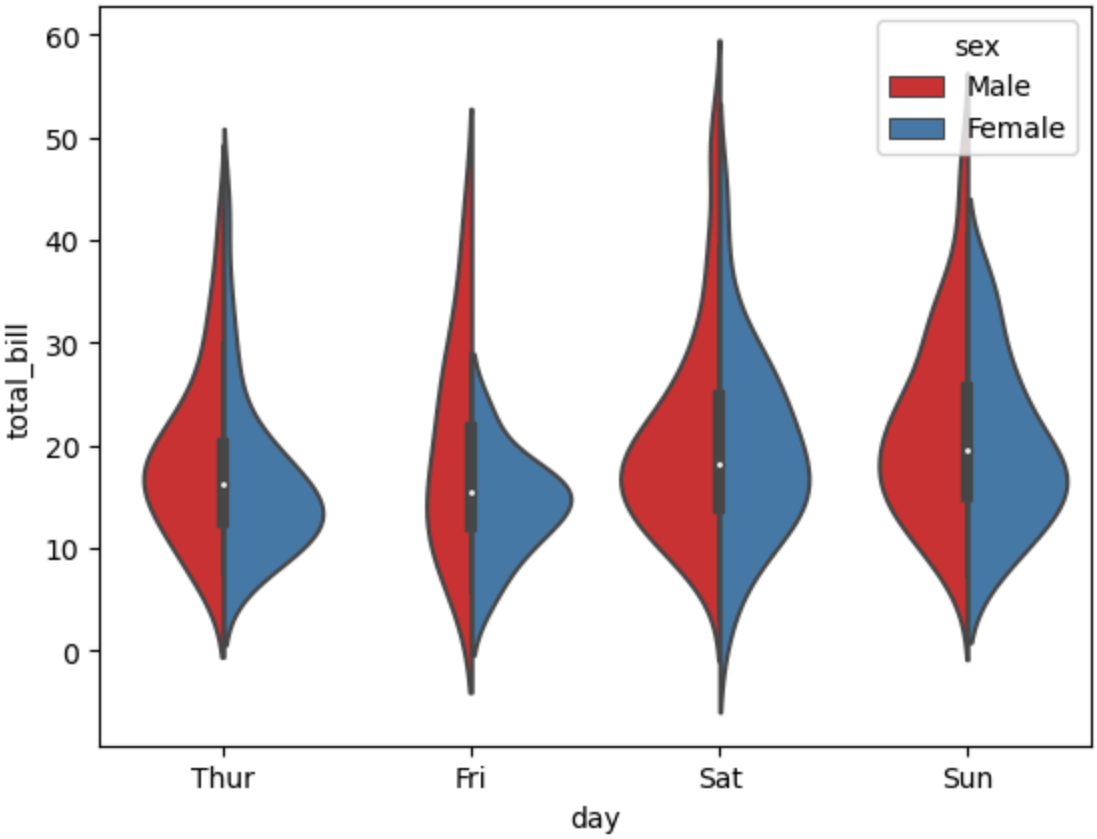
In [43]:
```
sns.violinplot(x="day", y="total_bill", data=tips,hue='sex',palette='Set1')
```

Out[43]: <Axes: xlabel='day', ylabel='total_bill'>



In [44]:
```
sns.violinplot(x="day", y="total_bill", data=tips,hue='sex',split=True,palette='Set
```

Out[44]:
```
<Axes: xlabel='day', ylabel='total_bill'>
```



Veoma informativna vizualizacija je `heatmap`. Ona zapravo predočava korelacionu matricu u domen grafikona. Dajmo korelacionu matricu nad numeričkim podacima iz `tips` dataframe-a i iscrtajmo heatmap-u na osnovu nje.

In [49]:
```
tips_num = tips.select_dtypes(exclude=["category"])
```

In [50]:
```
tips_num.head()
```

Out[50]:

	total_bill	tip	size
0	16.99	1.01	2
1	10.34	1.66	3
2	21.01	3.50	3
3	23.68	3.31	2
4	24.59	3.61	4

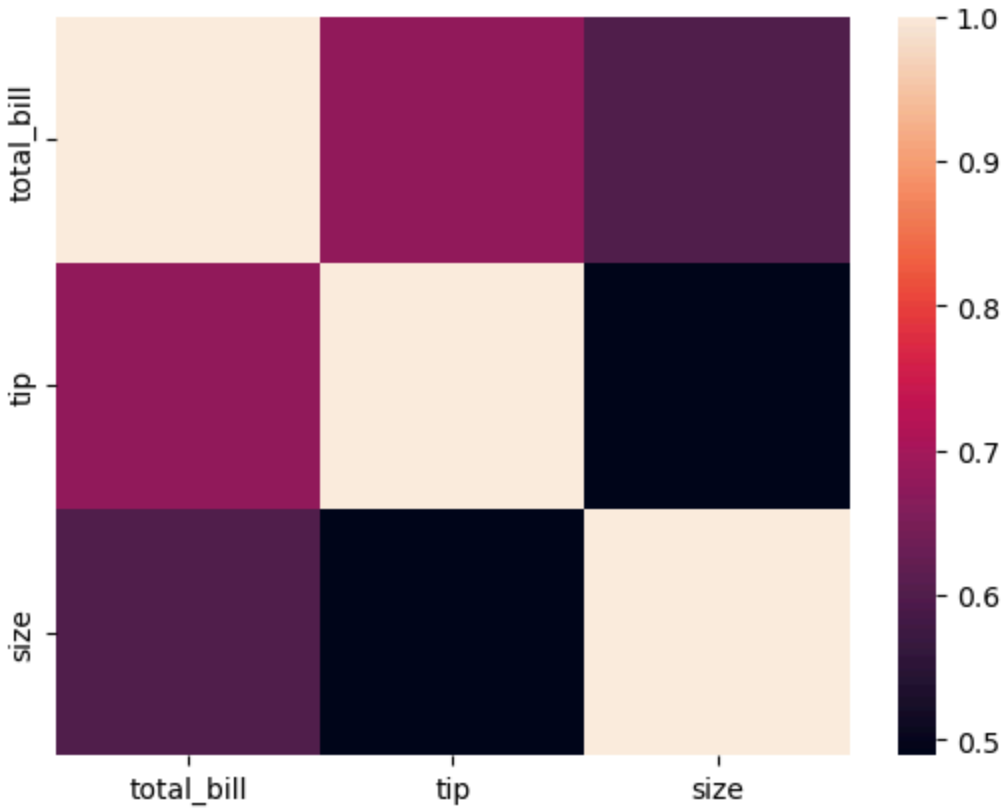
In [52]:
```
tips_num.corr()
```

Out[52]:

	total_bill	tip	size
total_bill	1.000000	0.675734	0.598315
tip	0.675734	1.000000	0.489299
size	0.598315	0.489299	1.000000

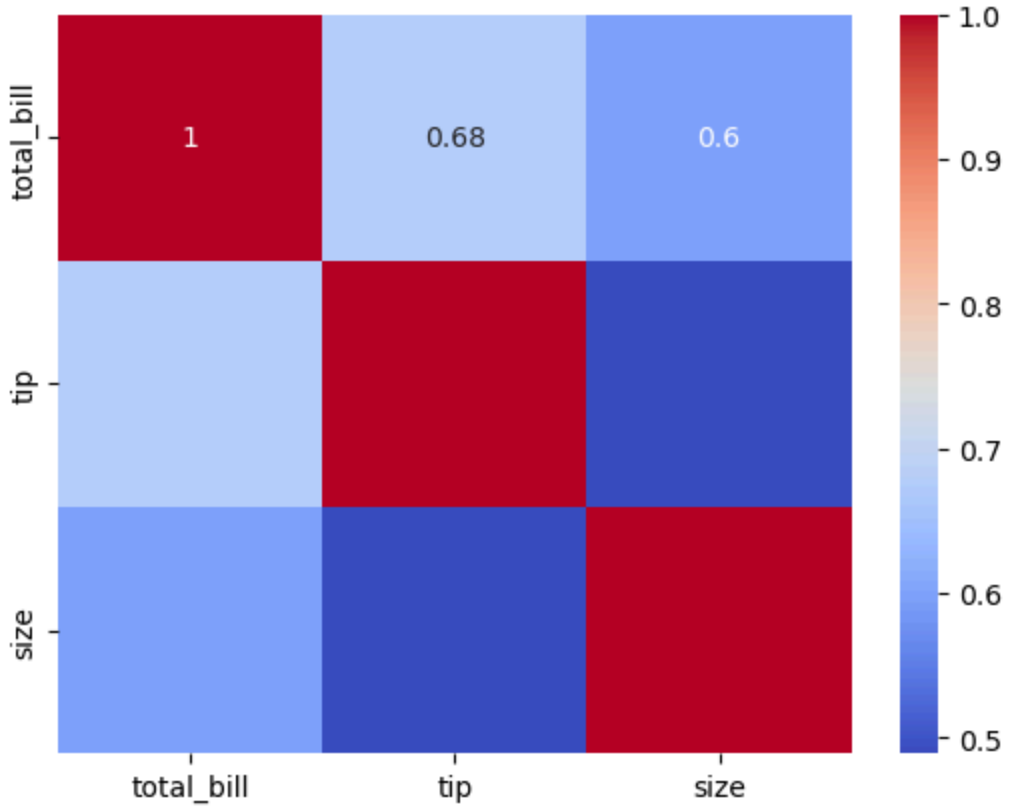
In [54]:
```
sns.heatmap(tips_num.corr())
```

Out[54]: <Axes: >



In [62]:
```
sns.heatmap(tips_num.corr(),cmap='coolwarm',annot=True)
```

Out[62]: <Axes: >


total_bill
tip
size
1
0.68
0.6
total_bill
tip
size
1.0
0.9
0.8
0.7
0.6
0.5